

AKADEMIA NAUK STOSOWANYCH W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA

Zespołowe przedsięwzięcie inżynierskie

System Maszynowego Liczenia Elementów

Autor:

Jakub Piwko
Łukasz Nowak
Agnieszka Ryś
Rafał Madoń

Prowadzący:

mgr inż. Nikodem Bulanda

Nowy Sącz 2025

Spis treści

1. Analiza zagadnienia	4
1.1. Wprowadzenie	4
1.2. Przykład zastosowania – SMT Reel Counter X800	4
2. Cel projektu	5
3. Wymagania funkcjonalne i нефункционалне	6
3.1. Wymagania funkcjonalne	6
3.2. Wymagania нефункционалне	6
4. Technologie:	7
4.1. POC	7
4.1.1. YOLO	8
4.1.2. Omówienie poszczególnych wykresów YOLO	11
4.2. Implementacja właściwa	14
4.2.1. Projekt na platformie online	14
4.2.2. Biblioteki i Frameworki	14
4.2.3. Sieci Neuronowe do Detekcji Obiektów	15
4.2.4. Proces Treningu	17
4.3. Użyte Algorytmy	18
4.3.1. Mask R-CNN	18
4.3.2. Faster R-CNN	19
4.3.3. MCNN (Multi-Column Convolutional Neural Network)	21
5. Diagram przypadku użycia	22
5.1. Scenariusze przypadków użycia:	23
6. Instrukcja Obsługi Aplikacji	35
6.1. Logowanie	35
6.2. Strona Zliczania	36
6.3. Strona Trenowania Modeli	37
6.4. Strona Modeli	38

6.5. Strona Oznaczania Zdjęć	39
6.6. Strona Tworzenia Datasetu	40
6.7. Strona Benchmark	41
6.8. Strona Użytkownicy	42
6.9. Strona Historia	43
7. Czasowa i Zasobowa Analiza Projektu	44
7.1. Wykresy Burnup Sprintów	44
7.2. Sposoby komunikacji	54
8. Przykład Implementacji - Trening	55
8.1. Trening - API	55
8.2. Przykład Implementacji Treningu Mask R-CNN	57
9. Przykład Implementacji - Zliczanie	60
9.1. Zliczanie - API	60
9.2. Przykład Implementacji Zliczania Mask R-CNN	64
Literatura	67
Spis rysunków	68
Spis tabel	69
Spis listingów	70

1. Analiza zagadnienia

1.1. Wprowadzenie

Zdjęcia rentgenowskie znajdują zastosowanie nie tylko w medycynie. Jednym z przykładów może być chociażby skan bagażu na lotnisku, aby upewnić się że nie ma tam żadnych niebezpiecznych przedmiotów czy substancji.

Rentgen znalazł zastosowanie także w przemyśle i jest stosowany dość często aby sprawdzić stan produkowanych rzeczy bez ingerencji w ich strukturę. Taki zabieg nazywany jest diagnostyką nieniszczącą. Za pomocą odpowiednio skonfigurowanych skanerów można sprawdzić zawartość pudeł czy taśm i stwierdzić czy ich stan odpowiada wymaganiom produkcyjnym.

1.2. Przykład zastosowania – SMT Reel Counter X800

Urządzenie **SMT Reel Counter X800** jest przykładem systemu, który łączy technologię zdjęć rentgenowskich z zaawansowanymi algorytmami przetwarzania obrazu do automatycznego liczenia elementów na rolkach. Choć nie jest to główny temat analizy, jego działanie ilustruje praktyczne zastosowanie omówionej metody:

- **Wysokiej jakości skanowanie:**

Urządzenie wykorzystuje precyzyjne kamery i odpowiednio skonfigurowane źródło promieniowania, co pozwala na uzyskanie obrazów z wysokim kontrastem i rozdzielczością.

- **Automatyczna analiza:**

Po wykonaniu zdjęcia, system automatycznie przetwarza obraz – od filtracji, przez segmentację, aż po detekcję i liczenie elementów, co pozwala na szybkie i precyzyjne ustalenie liczby komponentów na rolce.

- **Integracja z linią produkcyjną:**

Wyniki analizy są wykorzystywane w czasie rzeczywistym, umożliwiając bieżącą kontrolę jakości i automatyzację procesów produkcyjnych.

2. Cel projektu

Celem naszego projektu jest stworzenie oprogramowania, które umożliwi automatyczne zliczanie elementów znajdujących się na rolce z układami scalonymi po procesie produkcyjnym. System ten wykorzysta zdjęcia rentgenowskie oraz zaawansowane algorytmy oparte na uczeniu maszynowym w celu precyzyjnego określenia liczby komponentów na rolce.

Oprogramowanie będzie realizowało następujące funkcje:

- **Przetwarzanie obrazów rentgenowskich** – system będzie analizować zdjęcia rentgenowskie rolki, identyfikując i zliczając układy scalone.
- **Zastosowanie algorytmów uczenia maszynowego** – wykorzystanie modeli ML do poprawy dokładności detekcji i klasyfikacji elementów na podstawie dostarczonych danych.
- **Możliwość trenowania nowych modeli** – użytkownik będzie mógł dodawać nowe dane treningowe, co pozwoli na dostosowanie systemu do różnych typów komponentów oraz warunków skanowania.
- **Prezentacja wyników** – system dostarczy użytkownikowi czytelne raporty i wizualizacje dotyczące liczby wykrytych elementów na analizowanej rolce.
- **Integracja z procesem produkcyjnym** – możliwość wykorzystania wyników analizy do automatyzacji kontroli jakości oraz monitorowania procesu pakowania układów scalonych.

Dzięki tym funkcjom system zapewni wysoką precyzję w zliczaniu elementów, co pozwoli na eliminację błędów ludzkich oraz zwiększenie efektywności kontroli jakości w procesie produkcji układów scalonych.

3. Wymagania funkcjonalne i нефункционалне

3.1. Wymagania funkcjonalne

- Możliwość wczytywania obrazów przez użytkownika.
- Wybór operacji na załadowanym obrazie (analiza lub trenowanie modelu).
- Analiza obrazu i zliczanie elementów przy użyciu modelu.
- Prezentowanie wyników analizy użytkownikowi.
- Możliwość uczenia nowego modelu na podstawie oznaczonych danych.
- Możliwość tworzenia własnych datasetów przez użytkownika.
- Automatyczne labelowanie obrazów przy użyciu wcześniej nauczonego modelu.
- Możliwość ręcznego labelowania zdjęć w aplikacji.
- Możliwość preprocessingu zdjęć przed analizą.
- Archiwizacja akcji wykonywanych przez użytkowników.

3.2. Wymagania нефункционалне

- Interfejs użytkownika powinien być intuicyjny i łatwy w obsłudze.
- System powinien zapewniać wysoką dokładność wykrywania elementów.
- Dane użytkowników i modele powinny być przechowywane w sposób bezpieczny.
- System powinien wspierać różne formaty obrazów.
- Operacje preprocessingu i labelowania powinny być wykonywane w akceptowalnym czasie.

4. Technologie:

4.1. POC

YOLO (You Only Look Once) to jedna z najpopularniejszych sieci neuronowych do detekcji obiektów w czasie rzeczywistym. Charakteryzuje się dużą szybkością i dokładnością dzięki jednorazowemu przetwarzaniu obrazu.

Algorytm YOLO został po raz pierwszy zaprezentowany przez Redmona i in. w 2016 roku [1]. Jest to jednofazowy detektor obiektów, który przetwarza obraz w czasie rzeczywistym, osiągając wysoką precyzję i szybkość działania. W projekcie wykorzystano jego najnowszą wersję – YOLOv8, udostępnioną przez zespół Ultralytics [2].

OpenCV (Open Source Computer Vision Library) to biblioteka służąca do przetwarzania obrazu i analizy wizyjnej. Pozwala na integrację z YOLO w celu detekcji i śledzenia obiektów na obrazach i wideo.

Zastosowanie

Połączenie YOLO i OpenCV umożliwia:

- Wykrywanie obiektów w czasie rzeczywistym
- Analizę obrazu i wideo
- Rozpoznawanie twarzy i tablic rejestracyjnych
- Wykorzystanie w systemach bezpieczeństwa i autonomicznych pojazdach

4.1.1. YOLO

Do implementacji uczenia maszynowego na podstawie modelu użyto algorytmu YOLO w wersji 8. Do treningu oraz detekcji obiektów zastosowano bibliotekę udostępnioną przez firmę Ultralytics, zawierającą predefiniowane modele YOLOv8 oraz narzędzia do ich konfiguracji i testowania [2]. Dokumentacja biblioteki dostępna jest online (dostęp: 20.05.2025).

```

1 import argparse
2 from ultralytics import YOLO
3 import os
4
5 def main(args):
6     model = YOLO(args.model)
7
8     # Trenujemy model na danych określonych w pliku data.yaml
9     model.train(data=args.data, epochs=args.epochs, imgsz=args
    .imgsz)
10    print("Trening zakończony. Wyniki zapisane w folderze runs
    /train/weights")
11
12 if __name__ == "__main__":
13     parser = argparse.ArgumentParser(description="Trenowanie
    modelu YOLO na zbiorze SMD.")
14     parser.add_argument("--model", type=str, default="../
    yolov8n.pt", help="Ścieżka do pretrenowanego modelu YOLO (
    np. yolov8n.pt)")
15     parser.add_argument("--data", type=str, default="../
    datasets/my_smd_dataset/data.yaml", help="Ścieżka do pliku
    data.yaml opisującego zbiór danych")
16     parser.add_argument("--epochs", type=int, default=50, help
    ="Liczba epok treningowych")
17     parser.add_argument("--imgsz", type=int, default=640, help
    ="Rozmiar obrazu podczas treningu")
18     args = parser.parse_args()
19     main(args)

```

Listing 1. Kod odpowiedzialny za trenowanie

Na listingu 6.3 przedstawiono kod odpowiedzialny za wyuczenie konkretnego modelu.


```

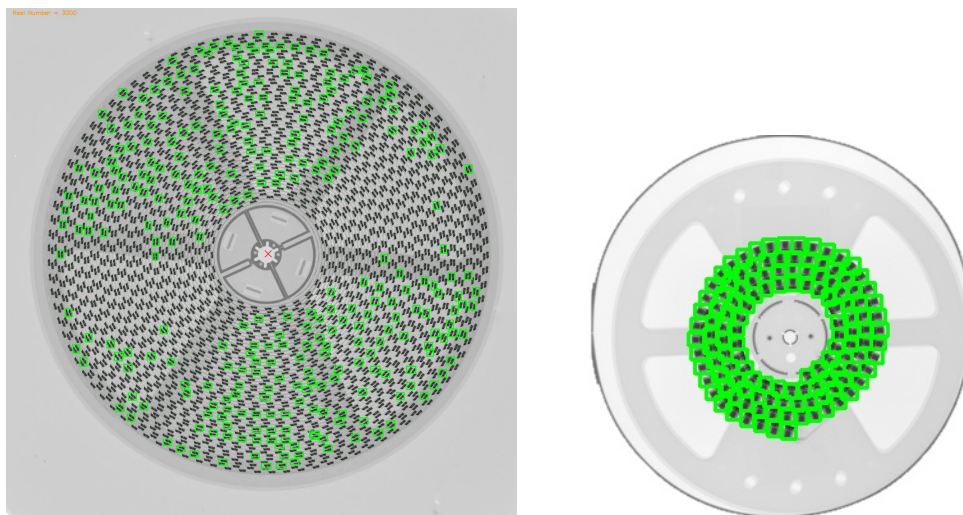
1 import argparse
2 import cv2
3 from ultralytics import YOLO
4
5 def main(args):
6     model = YOLO(args.weights)
7
8     # Wykonujemy detekcję na obrazie
9     results = model(args.source)
10
11     # Dla każdego obrazu (w tym przypadku jeden obraz)
12     zapisujemy wynikową anotację
13     for idx, result in enumerate(results):
14         img = result.orig_img # Originalny obraz
15         for box in result.bboxes.xyxy: # Przechodzimy przez
16             wszystkie bounding boxy
17             x1, y1, x2, y2 = map(int, box) # Współrzędne boxa
18             cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0)
19             , 2) # Rysowanie boxa
20
21     # Zapisujemy obraz z boxami
22     output_path = f"annotated_{idx}.jpg"
23     cv2.imwrite(output_path, img)
24     print(f"Wykryto {len(result.bboxes)} obiektów na
25     obrazie {args.source}. Wynik zapisany jako {output_path}")
26
27 if __name__ == "__main__":
28     parser = argparse.ArgumentParser(description="Detekcja
29     obiektów przy użyciu wytrenowanego modelu YOLO.")
30     parser.add_argument("--weights", type=str, default="../
31     best.pt", help="Ścieżka do wytrenowanego modelu (np. best.
32     pt)")
33     parser.add_argument("--source", type=str, default="../
34     spula.jpg", help="Ścieżka do obrazu wejściowego")
35     args = parser.parse_args()
36     main(args)

```

Listing 2. Kod odpowiedzialny za znajdowanie elementów

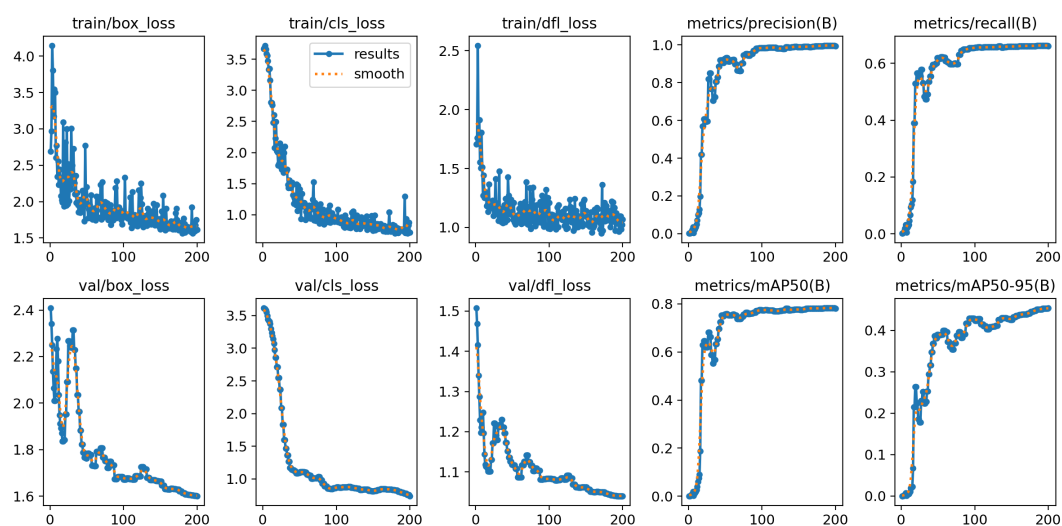
Na listingu 2 przedstawiono kod odpowiedzialny za znajdowanie oraz liczenie elementów na podstawie wyuczonego modelu.

Na obrazkach z numerem 4.1 przedstawiono wyniki wyszukiwania elementów po nauce tylko z obrazka po prawej stronie. Skuteczność bez uczenia sprawdzono



Rys. 4.1. Oznaczenie wykrytych elementów

na obrazku po lewej, gdzie widać słabe efekty wykrywania, z pominięciem ponad połowy elementów.



Rys. 4.2. Wykres skuteczności uczenia modelu

Na obrazku 4.2 przedstawiono wykresy dotyczące nauki modelu przez algorytm YOLO. Wy tłumaczenie poszczególnych wykresów przedstawiono poniżej:

4.1.2. Omówienie poszczególnych wykresów YOLO

Poniżej znajduje się omówienie poszczególnych wykresów przedstawionych na ilustracji (typowo generowanych przez YOLO podczas treningu). W większości przypadków wyświetlane są dwie krzywe:

- **results** – surowe wyniki z każdej epoki,
- **smooth** – te same wyniki poddane wygładzaniu (ułatwia dostrzeżenie trendu).

1. `train/box_loss`

- **Co mierzy?**

Jest to strata związana z dopasowaniem prognozowanego położenia (bounding box) obiektu do jego faktycznej lokalizacji w obrazie. Im mniejsza wartość, tym lepiej model „nauczył się” określać prawidłowe wymiary i położenie detekcji.

- **Interpretacja wykresu:**

Stopniowy spadek `box_loss` w trakcie treningu świadczy o tym, że model coraz trafniej przewiduje bounding boxy dla obiektów na obrazach treningowych.

2. `train/cls_loss`

- **Co mierzy?**

Odpowiada za stratę związaną z klasyfikacją wykrytych obiektów do właściwej kategorii (*classification loss*). Dla każdego bounding boxa model próbuje określić, do której klasy należy obiekt.

- **Interpretacja wykresu:**

Spadek `cls_loss` oznacza, że model coraz lepiej rozpoznaje, jaki rodzaj obiektu znajduje się w danym bounding boxie na zbiorze treningowym.

3. `train/df1_loss`

- **Co mierzy?**

Jest to tzw. *Distribution Focal Loss*, stosowana w YOLOv8 (i nowszych podejściach) do bardziej precyzyjnego dopasowania krawędzi bounding boxów. Rozszerza ona klasyczny *focal loss*, by lepiej uwzględnić rozkład prawdopodobieństwa brzegów detekcji.

- **Interpretacja wykresu:**

Spadek `df1_loss` pokazuje, że model skuteczniej uczy się dokładnego lokalizowania granic obiektów (np. poprawia precyzję w określaniu lewego, prawego, górnego i dolnego brzegu boxa).

4. `val/box_loss`

- **Co mierzy?**

Analogicznie do `train/box_loss`, ale obliczane na zbiorze walidacyjnym. Pozwala sprawdzić, czy model nie przeucza się (*overfitting*) i czy potrafi generalizować na danych niewidzianych w treningu.

- **Interpretacja wykresu:**

Powinno zachodzić podobne zjawisko spadku wartości co w przypadku treningu, jednak z reguły wykres może być mniej „gładki” i nieco wyższy niż `train/box_loss`.

5. `val/cls_loss`

- **Co mierzy?**

Stratę klasyfikacji (*classification loss*) na zbiorze walidacyjnym. Określa, jak dobrze model przypisuje poprawną klasę wykrytym obiektom poza zestawem treningowym.

- **Interpretacja wykresu:**

Spadek wartości świadczy o poprawnej generalizacji modelu w zakresie rozróżniania klas. Jeśli wartości są dużo wyższe niż na zbiorze treningowym, może to wskazywać na przeuczenie.

6. `val/dfl_loss`

- **Co mierzy?**

Distribution Focal Loss na zbiorze walidacyjnym. Podobnie jak w przypadku `train/dfl_loss`, pokazuje, na ile model potrafi precyzyjnie określać granice obiektów w nieznanach wcześniej próbkach.

- **Interpretacja wykresu:**

Równomierny spadek świadczy o tym, że model poprawnie lokalizuje obiekty również na danych walidacyjnych. Zbyt duża różnica między `train/dfl_loss` a `val/dfl_loss` może wskazywać na problemy z uogólnieniem.

7. `metrics/precision(B)`

- **Co mierzy?**

Precision (precyzja) na zbiorze walidacyjnym, czyli odsetek wykrytych obiektów, które faktycznie są poprawne. W kontekście detekcji obiektów oznacza to, jaki procent wykryć jest prawidłowy (tj. ma odpowiedni IoU z obiektem rzeczywistym).

- **Interpretacja wykresu:**

Wzrost precyzji oznacza, że model rzadziej „fałszywie” rozpoznaje obiekty (mniej *false positives*). Jeśli wykres jest wysoko (blisko 1.0), to znaczy, że większość zidentyfikowanych obiektów jest poprawna.

8. `metrics/recall(B)`

- **Co mierzy?**

Recall (czułość) na zbiorze walidacyjnym, czyli jaki odsetek rzeczywistych obiektów został poprawnie wykryty przez model.

- **Interpretacja wykresu:**

Wzrost *recall* oznacza, że model pomija coraz mniej obiektów (mniej *false negatives*). Gdy *recall* jest wysoki (blisko 1.0), model wykrywa niemal wszystkie obiekty, jednak czasem może to się odbywać kosztem większej liczby fałszywych detekcji.

(Opcjonalnie) metrics/mAP50(B) i metrics/mAP50-95(B)

Choć nie ma ich na wszystkich zestawieniach, w niektórych raportach YOLO widać też:

- **mAP@0.5** – *mean Average Precision* przy $\text{IoU} = 0.5$, czyli średnia precyzja uśredniona po wszystkich klasach, gdy za poprawne wykrycie uznaje się box nachodzący na ground truth w co najmniej 50%.
- **mAP@0.5:0.95** – to bardziej restrykcyjna metryka, gdzie oblicza się mAP dla wielu progów IoU (od 0.5 do 0.95 w krokach co 0.05) i uśrednia wynik. Jest trudniejsza do spełnienia, ale lepiej oddaje faktyczną skuteczność modelu w różnych warunkach.

4.2. Implementacja właściwa

4.2.1. Projekt na platformie online

Całość projektu została przeprowadzona w grupie czteroosobowej. Kod był utrzymywany za pomocą systemu kontroli wersji **Git** [3], a repozytorium hostowane było na platformie **GitHub** [4].

Współpraca ta polegała na dzieleniu się częściami projektu (kodu), które mogły być implementowane niezależnie, co pozwoliło znacznie skrócić czas realizacji prac. Dzięki wykorzystaniu systemu kontroli wersji (VCS, ang. *Version Control System*) możliwe było zapisywanie postępów, eksperymentowanie z różnymi podejściami oraz zachowanie historii zmian bez ryzyka utraty kodu.

GitHub umożliwił nie tylko bezpieczne przechowywanie kodu, ale również jego wygodne udostępnianie pozostałym członkom zespołu, co usprawniło integrację poszczególnych elementów systemu.

Cały projekt jest publicznie dostępny w repozytorium:

<https://github.com/Jaciesune/SMLE>

4.2.2. Biblioteki i Frameworki

TensorFlow – popularny framework do głębokiego uczenia, obsługujący modele detekcji obiektów. Dzięki rozbudowanemu ekosystemowi narzędzi, TensorFlow umożliwia łatwe tworzenie, trenowanie oraz wdrażanie modeli sieci neuronowych w zastosowaniach przemysłowych.

PyQt5 - jest to zestaw narzędzi umożliwiających tworzenie graficznych interfejsów użytkownika (GUI) w języku Python, oparty na frameworku Qt. PyQt5 pozwala na szybkie i efektywne budowanie aplikacji desktopowych o nowoczesnym wyglądzie, oferując szeroki wachlarz widżetów, takich jak przyciski, okna dialogowe, paski narzędzi, menu, tabele i inne. W kontekście naszego projektu, PyQt5 może być wykorzystany do:

- **Tworzenia interfejsu użytkownika:** Umożliwia zaprojektowanie intuicyjnego interfejsu, w którym użytkownik może łatwo wprowadzać dane, np. załadować obrazy do analizy, wybierać modele czy przeglądać wyniki detekcji.
- **Wizualizacji wyników:** Dzięki PyQt5 można dynamicznie wyświetlać wyniki działania modeli detekcji, np. obrazy z nałożonymi bounding boxami lub maskami segmentacyjnymi, co ułatwia analizę i ocenę jakości detekcji.
- **Integracji z backendem:** Interfejs stworzony za pomocą PyQt5 może być bezproblemowo połączony z backendem opartym na TensorFlow i Keras, umożliwiając przetwarzanie danych w czasie rzeczywistym i natychmiastowe przedstawienie wyników użytkownikowi.
- **Obsługi zdarzeń:** PyQt5 oferuje mechanizmy obsługi zdarzeń (takich jak kliknięcia, przesunięcia myszą, wpisywanie danych), co pozwala na płynne sterowanie aplikacją i reagowanie na interakcje użytkownika.

Dzięki wykorzystaniu PyQt5, projekt zyskuje na użyteczności i intuicyjności, co jest kluczowe w zastosowaniach przemysłowych, gdzie szybka i przejrzysta prezentacja wyników oraz interakcja z systemem są niezwykle istotne.

4.2.3. Sieci Neuronowe do Detekcji Obiektów

Mask R-CNN to rozszerzenie popularnej architektury Faster R-CNN, które oprócz detekcji obiektów umożliwia także segmentację instancyjną – czyli precyzyjne określenie kształtu obiektów.

W środowisku **Keras** (często używanym w połączeniu z TensorFlow) dostępne są implementacje Mask R-CNN, które umożliwiają:

- **Dokładniejszą segmentację:** Mask R-CNN generuje maski dla każdego wykrytego obiektu, co pozwala na bardziej precyzyjne określenie jego konturów.

- **Łatwą integrację z pipeline'ami deep learningowymi:** Dzięki wysokopoziomowemu API Keras, można szybko eksperymentować z różnymi modelami oraz dostosowywać architekturę do specyficznych potrzeb projektu.
- **Wsparcie dla transfer learningu:** Można korzystać z pretrenowanych modeli, co znacznie przyspiesza proces uczenia oraz zwiększa skuteczność detekcji i segmentacji.

4.2.4. Proces Treningu

Proces treningu modelu detekcji obiektów obejmuje kilka kluczowych etapów:

1. **Przygotowanie danych:**

Zbiór danych musi być starannie przygotowany i odpowiednio oznaczony. Użycie pliku konfiguracyjnego (np. `data.yaml`) umożliwia zdefiniowanie lokalizacji obiektów w obrazach, co jest kluczowe dla procesu uczenia.

2. **Konfiguracja modelu:**

Na etapie konfiguracji wybiera się pretrenowany model, który zostanie dostosowany do specyfiki nowego zbioru danych. Ustawienie hiperparametrów, takich jak liczba epok, rozmiar obrazu czy współczynnik uczenia, jest kluczowe dla optymalizacji procesu trenowania.

3. **Trening:**

Model jest trenowany na przygotowanym zbiorze danych, podczas którego następuje iteracyjne dostosowywanie wag sieci w celu minimalizacji funkcji straty. Regularne monitorowanie metryk, takich jak dokładność czy precyzja, umożliwia wczesne wykrycie ewentualnych problemów.

4. **Walidacja i testowanie:**

Po zakończeniu treningu, model jest testowany na niezależnym zbiorze walidacyjnym, co pozwala na ocenę jego wydajności na danych niewidzianych podczas procesu trenowania.

5. **Wdrożenie:**

Wytrenowany model może być zintegrowany z systemem produkcyjnym, gdzie przetwarza obrazy w czasie rzeczywistym, umożliwiając automatyczną detekcję i liczenie obiektów.

4.3. Użyte Algorytmy

4.3.1. Mask R-CNN

Mask R-CNN to architektura zaprezentowana przez He i in. w 2017 roku, która rozszerza model Faster R-CNN o dodatkową głowicę segmentacyjną, pozwalającą na pikselową detekcję obiektów [5]. Dzięki zastosowaniu warstwy RoIAlign uzyskano większą precyzję konturów segmentacji w porównaniu do wcześniejszych podejść.

Model składa się z trzech głównych etapów:

- **Ekstrakcja cech** — z wykorzystaniem głębokiej sieci konwolucyjnej (np. *ResNet*, *ResNeXt*),
- **Generowanie propozycji regionów** (Region Proposal Network — *RPN*),
- **Predykcja trzech elementów** dla każdego regionu:
 - klasy obiektu,
 - ramki ograniczającej (*bounding box*),
 - maski segmentacji.

Architektura opiera się na sieci **Feature Pyramid Network (FPN)**, która umożliwia detekcję obiektów o różnych rozmiarach. Kluczowym ulepszeniem względem Faster R-CNN jest wprowadzenie modułu „**mask head**”, który przewiduje maski binarne dla każdego wykrytego obiektu.

Aby poprawić jakość segmentacji, zamiast klasycznego *RoIPool*, Mask R-CNN stosuje **RoIAlign**, który eliminuje błędy kwantyzacji i zapewnia lepsze dopasowanie masek do rzeczywistych kształtów obiektów.

Zastosowania:

- diagnostyka medyczna,
- analiza biologiczna,
- segmentacja obiektów w pojazdach autonomicznych.

Choć Mask R-CNN ma wyższe wymagania obliczeniowe niż Faster R-CNN, rekompensuje to większą precyzją i możliwością analizy struktury obiektów na obrazie.

4.3.2. Faster R-CNN

Faster R-CNN to jedna z najbardziej uznanych architektur detekcji obiektów w obrazach, łącząca wysoką precyzję z rozsądną szybkością działania. Stanowi rozwinięcie wcześniejszych modeli *R-CNN* oraz *Fast R-CNN*, wprowadzając kluczową innowację — zintegrowaną sieć generującą propozycje regionów zainteresowania, czyli **Region Proposal Network (RPN)**. Dzięki temu eliminuje konieczność stosowania zewnętrznych metod takich jak *Selective Search*.

Faster R-CNN został opracowany przez zespół Shaoqinga Rena i zaprezentowany w 2015 roku jako ewolucja modeli R-CNN oraz Fast R-CNN [6].

Model składa się z dwóch głównych komponentów:

- **Sieć konwolucyjna** (np. *VGG16*, *ResNet*) — odpowiada za ekstrakcję cech z obrazu,
- **Region Proposal Network (RPN)** — analizuje mapy cech i generuje propozycje regionów, w których mogą znajdować się obiekty.

Dla każdego zaproponowanego regionu wykonywane są dwa działania:

- **Klasyfikacja** — określenie, do jakiej klasy należy obiekt,
- **Regresja ramki ograniczającej** — dopasowanie pozycji i rozmiaru obiektu w obrazie.

Faster R-CNN działa w trybie **dwustopniowym**:

1. najpierw lokalizuje potencjalne obszary z obiektami,
2. następnie dokonuje ich analizy i klasyfikacji.

W porównaniu do algorytmów jednoetapowych (np. *YOLO*), Faster R-CNN oferuje większą precyzję — zwłaszcza w przypadku trudnych do wykrycia obiektów lub złożonych scen. Wadą może być mniejsza szybkość działania, co jednak rekompensuje jakością wyników.

Zastosowania:

- systemy monitoringu,
- rozpoznawanie twarzy,

- analiza obrazów medycznych,
- robotyka i systemy autonomiczne.

4.3.3. MCNN (Multi-Column Convolutional Neural Network)

MCNN, czyli *Multi-Column Convolutional Neural Network*, to architektura zaprojektowana specjalnie do zadań związanych ze **zliczaniem obiektów na obrazach** — szczególnie w scenach o dużym zagęszczeniu i nachodzeniu się obiektów, takich jak tłumy ludzi, ruch uliczny czy zgromadzenia.

W odróżnieniu od detektorów takich jak *Faster R-CNN* czy *Mask R-CNN*, które lokalizują i klasyfikują obiekty, MCNN generuje tzw. **mapę gęstości (density map)**. Na jej podstawie szacowana jest liczba obiektów w obrazie — nie przez wykrywanie pojedynczych elementów, ale poprzez analizę zagęszczenia w różnych częściach obrazu. *Suma wartości pikseli na mapie odpowiada liczbie obiektów.*

Najważniejszą cechą MCNN jest wykorzystanie wielu **równoległych kolumn konwolucyjnych**, z których każda operuje na filtrach o innych rozmiarach. Pozwala to na:

- skuteczne rozpoznawanie obiektów o różnych skalach,
- radzenie sobie z perspektywą i zmienną wielkością obiektów (np. osoby blisko i daleko od kamery),
- lepsze dopasowanie do rzeczywistych warunków zdjęć.

Zalety MCNN:

- mniejsze wymagania obliczeniowe w porównaniu z klasycznymi detektorami,
- brak potrzeby dokładnego oznaczania każdego obiektu,
- duża skuteczność w estymacji liczby obiektów nawet w bardzo zatłoczonych scenach.

Zastosowania MCNN:

- analiza tłumów i demonstracji,
- zliczanie pojazdów w ruchu drogowym,
- monitoring przestrzeni publicznych,
- zliczanie drzew, zwierząt lub innych obiektów na zdjęciach satelitarnych.

5.1. Scenariusze przypadków użycia:

Tab. 5.1. Scenariusz 1 - Rejestracja

Nazwa	Rejestracja konta w systemie
Aktor	Administrator
Warunki początkowe	• Admin jest zalogowany w systemie.
Opis	Administrator dodaje nowego użytkownika do systemu.
Ścieżka główna	1. Administrator wybiera opcje "Utwórz nowego użytkownika" w panelu "Lista użytkowników". 2. System przenosi administratora na stronę rejestracyjną. 3. Administrator wypełnia wymagane pola ustawiając hasło wygenerowane komputerowo. 4. Po wypełnieniu wymaganych pól administrator klika przycisk "Utwórz użytkownika". 5. System przeprowadza validacje. 6. Nowo utworzony użytkownik zostaje dodany do bazy danych.
Warunki końcowe	• Konto zostaje dodane do bazy • W przypadku niepowodzenia system wyświetla komunikat o błędzie.

Tab. 5.2. Scenariusz 2 - Logowanie

Nazwa	Logowanie
Aktor	Gość
Warunki początkowe	• Gość nie jest zalogowany do systemu. • Użytkownik (gość) posiada dane uwierzytelniające (login, hasło). • Konto istnieje w bazie danych.
Opis	Gość wprowadza swoje dane logowania, które następnie są weryfikowane przez system.
Ścieżka główna	1. Gość wybiera opcję <i>Zaloguj się</i>. 2. System wyświetla formularz logowania (pole na login i hasło). 3. Gość wypełnia formularz i zatwierdza. 4. System weryfikuje dane (walidacja). 5. System przenosi gościa do strony głównej w przypadku poprawnych danych.
Warunki końcowe	• Gość staje się zalogowanym użytkownikiem i ma dostęp do funkcji systemu przeznaczonych dla zalogowanych. • W przypadku niepowodzenia system wyświetla komunikat o błędzie. • W przypadku dezaktywowanego konta przez administratora wyświetla się komunikat o braku możliwości logowania.

Tab. 5.3. Scenariusz 3 - Trenowanie nowego modelu

Nazwa	Trenowanie nowego modelu
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie
Opis	Użytkownik po zalogowaniu do systemu wybiera opcję "Trening", aby wytrenować nowy model do systemu.
Ścieżka główna	1. Użytkownik wybiera opcję <i>Nowy Model</i>. 2. System przenosi użytkownika do panelu trenowania modelu. 3. Użytkownik wprowadza nazwę modelu w polu "Nazwa modelu". 4. Użytkownik wybiera algorytm dostępny z listy. 5. Użytkownik dodaje dataset do programu, na którego podstawie zostanie nauczony model. 6. Po wypełnieniu wymaganych pól użytkownik kończy proces trenowania nowego modelu przyciskiem "Rozpocznij trening". 7. Po utworzeniu modelu system automatycznie uruchamia proces pierwszego nauczania modelu na bazie datasetu dodanego przez użytkownika.
Warunki końcowe	• Model zostaje dodany do listy modeli. • Użytkownik zostaje przeniesiony na stronę główną. • W przypadku niepowodzenia proces uczenia zostaje przerwany a błąd wyświetlony.

Tab. 5.4. Scenariusz 4 - Douczanie modelu

Nazwa	Douczanie istniejącego modelu
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie. • Model znajduje się w systemie.
Opis	Użytkownik po zalogowaniu do systemu wybiera opcję "Lista Modeli", po czym zostaje przeniesiony na stronę wyświetlającą istniejące modele w systemie.
Ścieżka główna	1. Użytkownik klika zębatkę z etykietą "Opcje", po czym klika na opcję "Ducz" z wysuwającego się menu. 2. Użytkownik zostaje przeniesiony na stronę "Ducz model" przez system. 3. Użytkownik ma możliwość dodania zdjęcia do douczania modelu, w przypadku pozostawienia pustego pola, program uczy model na wcześniej dodanych zdjęciach. 4. Użytkownik klika przycisk "Rozpocznij douczanie". 5. System przeprowadza proces douczania, następnie przenosi go na stronę główną.
Warunki końcowe	• Użytkownik znajduje się na stronie głównej.

Tab. 5.5. Scenariusz 5 - Importowanie modelu

Nazwa	Importowanie modelu
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie. • Plik z wytrenowanym modelem znajduje się na dysku użytkownika.
Opis	Użytkownik po zalogowaniu do systemu przechodzi do zakładki „Modele”, gdzie ma możliwość zaimportowania własnego modelu do systemu.
Ścieżka główna	1. Użytkownik przechodzi do zakładki „Modele” z menu nawigacyjnego. 2. Użytkownik klika przycisk „Importuj Model”. 3. System wyświetla formularz importu modelu. 4. Użytkownik wybiera z rozwijanej listy algorytm, z którym powiązany jest importowany model (np. Faster R-CNN, Mask R-CNN, MCNN). 5. Użytkownik wskazuje plik modelu zapisany lokalnie na dysku. 6. Użytkownik zatwierdza przyciskiem „Importuj”. 7. System zapisuje model w bazie danych oraz rejestruje go w aplikacji jako dostępny do użycia. 8. Użytkownik zostaje przeniesiony z powrotem do listy modeli, gdzie pojawia się nowo zaimportowany model.
Warunki końcowe	• Model jest dostępny w systemie i gotowy do użycia. • Zostaje powiązany z wybranym algorytmem.

Tab. 5.6. Scenariusz 6 - Usuwanie modelu

Nazwa	Usuwanie modelu z systemu
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie. • Model znajduje się w systemie. • Model został dodany przez użytkownika.
Opis	Użytkownik po zalogowaniu do systemu wybiera opcję "Lista Modeli", po czym zostaje przeniesiony na stronę wyświetlającą istniejące modele w systemie.
Ścieżka główna	• Użytkownik klika zębatkę z etykietą "Opcje", po czym klika na opcję "Usuń Model". • Model zostaje usunięty z dysku i jego status w bazie danych zostaje archiwizowany. Lista zostaje odświeżona.
Warunki końcowe	• Użytkownik znajduje się na stronie "Lista Modeli". • Jeśli użytkownik nie jest właścicielem modeli (model, na którym użytkownik chce przeprowadzić operację, nie został przez niego dodany), system wyświetli komunikat o nieposiadaniu uprawnień do przeprowadzenia operacji.

Tab. 5.7. Scenariusz 7 - Zliczanie elementów na podstawie modelu

Nazwa	Liczenie elementów na obrazie
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie. • Model znajduje się w systemie. • Obraz z którego elementy mają zostać zliczone został przesłany.
Opis	Użytkownik po zalogowaniu do systemu wybiera opcję "Zlicz Elementy", po czym zostaje przeniesiony na stronę "Zlicz Elementy".
Ścieżka główna	1. Użytkownik dodaje zdjęcie, z którego elementy mają zostać zliczone. 2. Użytkownik klika przycisk "Rozpocznij liczenie", uruchamiający proces. 3. System analizuje obraz. 4. System wybiera najbardziej pasujący model i rozpoczyna liczenie. 5. System przenosi użytkownika do strony "Wynik". 6. System wyświetla dane na temat modelu zastosowanego w liczeniu, liczbę elementów oraz czas operacji.
Warunki końcowe	• Użytkownik znajduje się na stronie "Wynik". • Jeśli model nie znajduje się w systemie wyświetlony zostaje komunikat o braku informacji o modelu i akcja zostaje przerwana. Użytkownik zostaje przeniesiony na stronę główną.

Tab. 5.8. Scenariusz 8 - Ustawienie statusu użytkownika

Nazwa	Ustawienie statusu użytkownika
Aktor	Administrator
Warunki początkowe	• Administrator jest zalogowany w systemie. • Użytkownik znajduje się w systemie. • Administrator znajduje się na stronie głównej.
Opis	Administrator może zmienić status użytkownika w systemie aby ten nie mógł się zalogować.
Ścieżka główna	1. Administrator wybiera przycisk "Zarządzanie", następnie z menu wysuwanego wybiera opcję "Lista użytkowników". 2. System przenosi administratora na odpowiednią stronę. 3. Administrator klika na zębatkę obok użytkownika, następnie klika opcję "Deaktywuj konto". 4. System zmienia status użytkownika na "Nieaktywne". 5. System uniemożliwia zalogowanie się nieaktywnemu użytkownikowi. 6. System odświeża listę użytkowników.
Warunki końcowe	• Użytkownik z nieaktywnym kontem zostanie odpowiednio poinformowany przy próbie logowania.

Tab. 5.9. Scenariusz 9 - Wyświetlenie historii operacji w systemie

Nazwa	Listowanie historii operacji
Aktor	Administrator
Warunki początkowe	• Administrator jest zalogowany w systemie. • Administrator znajduje się na stronie głównej.
Opis	Administrator może wyświetlić listę operacji przeprowadzanych w systemie z wyświetleniem danych o czasie, użytkownika oraz modelu przeprowadzonej operacji.
Ścieżka główna	1. Administrator wybiera przycisk "Zarządzanie", następnie z menu wysuwanego wybiera opcję "Historia". 2. System przenosi administratora na odpowiednią stronę. 3. System wyświetla listę operacji z danymi: - Użytego modelu - Użytego algorytmu - Typu operacji (dodanie modelu, liczenie, usunięcie modelu lub próba usunięcia) - Data wykonania operacji - Nazwa użytkownika wykonującego operację
Warunki końcowe	• Administrator może używać filtrowania podczas przeglądania strony.

Tab. 5.10. Scenariusz 10 - Tworzenie Datasetu

Nazwa	Tworzenie nowego datasetu
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie. • Użytkownik posiada lokalnie zapisane obrazy oraz odpowiadające im adnotacje (jeśli są dostępne).
Opis	Użytkownik przechodzi do zakładki „Tworzenie zbioru danych”, gdzie może załadować obrazy, przypisać je do odpowiednich podzbiorów (treningowy, walidacyjny, testowy) oraz powiązać z adnotacjami. Po zatwierdzeniu operacji system tworzy nowy dataset dostępny do dalszego użytku.
Ścieżka główna	1. Użytkownik wybiera z menu zakładkę „Tworzenie zbioru danych”. 2. System wyświetla formularz umożliwiający dodawanie zdjęć oraz adnotacji. 3. Użytkownik dodaje obrazy z dysku lokalnego. 4. Użytkownik dodaje odpowiadające pliki adnotacji. 5. Użytkownik przypisuje każde zdjęcie do jednej z kategorii: treningowy, walidacyjny, testowy. 6. Użytkownik nadaje nazwę dla zbioru danych. 7. Użytkownik klika przycisk „Utwórz dataset”. 8. System tworzy nowy dataset, zapisując obrazy, podziały oraz powiązane adnotacje. 9. Użytkownik zostaje przekierowany do listy dostępnych zbiorów danych, gdzie widnieje nowo utworzony zbiór.
Warunki końcowe	• Nowy dataset jest zapisany w systemie z kompletem danych i podziałem na zbiory. • Jest dostępny do trenowania i analizy modeli.

Tab. 5.11. Scenariusz 11 - Automatyczne oznaczanie zdjęć

Nazwa	Automatyczne oznaczanie zdjęć
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie. • W systemie istnieją obrazy do oznaczenia. • (Opcjonalnie) W systemie dostępny jest wcześniej wytrenowany model.
Opis	Użytkownik przechodzi do zakładki „Automatyczne oznaczanie zdjęć”, gdzie może wybrać metodę oznaczania: ręczną lub automatyczną przy użyciu dostępnego modelu. W przypadku automatycznego oznaczania użytkownik ma możliwość edycji i korekty oznaczeń, jeżeli zostaną wykryte błędy. Ostatecznie oznaczone obrazy mogą zostać zapisane jako część zbioru danych.
Ścieżka główna	1. Użytkownik wybiera zakładkę „Automatyczne oznaczanie zdjęć”. 2. System wyświetla listę dostępnych zdjęć do oznaczenia. 3. Użytkownik wybiera zdjęcia do oznaczenia. 4a. W przypadku oznaczania automatycznego – użytkownik wybiera model z listy, a system wykonuje automatyczne oznaczenie wybranych zdjęć. 5. Po zakończeniu oznaczania użytkownik ma możliwość edycji wyników i poprawy błędnych etykiet lub zaznaczeń. 6. Po weryfikacji oznaczeń użytkownik zapisuje zmiany. 7. System aktualizuje oznaczenia i przypisuje je do odpowiednich zdjęć w bazie danych.
Warunki końcowe	• Wybrane zdjęcia są poprawnie oznaczone (ręcznie lub automatycznie). • Adnotacje zostały zapisane w systemie i mogą być używane w datasetach.

Tab. 5.12. Scenariusz 12 - Benchmark modelu

Nazwa	Benchmark modelu
Aktor	Użytkownik, Administrator
Warunki początkowe	• Użytkownik jest zalogowany w systemie. • W systemie dostępny jest model do przetestowania. • Użytkownik posiada folder testowy zawierający zdjęcia oraz odpowiednie adnotacje.
Opis	Użytkownik przechodzi do zakładki „Benchmark”, wybiera model, który chce przetestować, a następnie wskazuje folder zawierający zdjęcia testowe oraz odpowiadające im adnotacje. System przeprowadza benchmark, porównując przewidywania modelu z adnotacjami, oblicza celność i zapisuje wynik w bazie danych dla tego modelu.
Ścieżka główna	1. Użytkownik wybiera zakładkę „Benchmark”. 2. System wyświetla dostępne modele w systemie. 3. Użytkownik wybiera model, który chce poddać benchmarkowi. 4. Użytkownik wskazuje folder testowy zawierający zdjęcia oraz adnotacje. 5. System rozpoczyna proces benchmarkowania, porównując wyniki modelu z adnotacjami. 6. System oblicza celność (accuracy) modelu na podstawie wyników porównania. 7. Po zakończeniu benchmarku system zapisuje wynik celności do bazy danych dla modelu. 8. Użytkownik otrzymuje informację o zakończeniu benchmarku oraz wartość celności modelu.
Warunki końcowe	• Celność modelu została obliczona i zapisana w systemie. • Wynik benchmarku jest dostępny w bazie danych przypisany do wybranego modelu.

6. Instrukcja Obsługi Aplikacji

Interfejs graficzny aplikacji desktopowej został zaimplementowany z użyciem biblioteki PyQt5 [8], która umożliwia tworzenie nowoczesnych, interaktywnych GUI w języku Python na podstawie frameworka Qt.

6.1. Logowanie

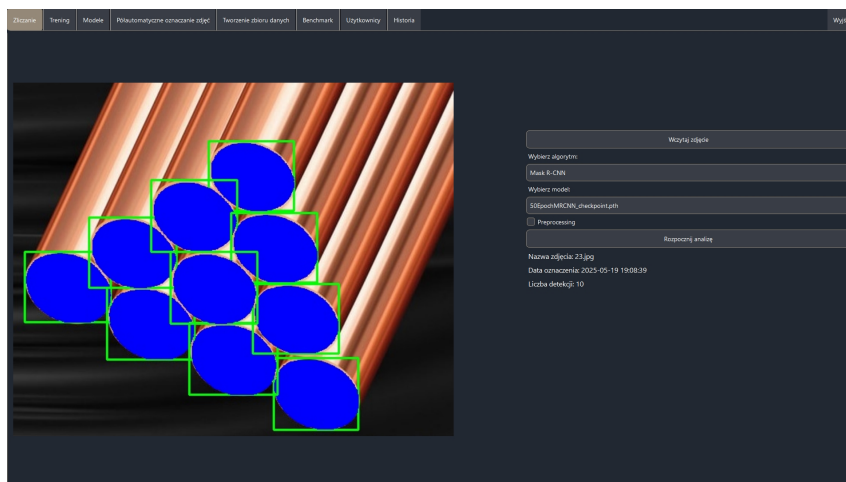
Po uruchomieniu aplikacji przed użytkownikiem wyświetlone zostaje okno logowania przedstawione na 6.1, które wymaga wprowadzenia nazwy oraz hasła użytkownika w celu zalogowania na odpowiednie konto.



Rys. 6.1. Formularz Logowania

6.2. Strona Zliczania

Po poprawnym zalogowaniu użytkownik zostaje przeniesiony na główną stronę jaką jest Strona Zliczania. Na niej użytkownik może wybrać: Zdjęcie, Algorytm, Model, a także Preprocessing w celu zliczania elementów na zdjęciu. Istnieje także Pasek Nawigacyjny u góry aplikacji, te funkcje przedstawione zostały na zdjęciu 6.2.



Rys. 6.2. Strona Zliczania

6.3. Strona Trenowania Modeli

Na **Stronie Trenowania Modeli** znajduje się formularz umożliwiający wytrenowanie nowego modelu lub douczenie już istniejącego.

Formularz zawiera szereg pól do wypełnienia, w tym:

- *Nazwa Modelu*,
- *Ścieżki do danych treningowych*,
- *Algorytm Modelu*,
- *Opcja Nowego Modelu lub Douczania*,
- *Liczba epok*,
- *Współczynnik Uczenia*,
- *Liczba Augmentacji*.

Po uzupełnieniu wszystkich wymaganych parametrów, proces trenowania można rozpocząć klikając przycisk „**Rozpocznij Trening**”, a w razie potrzeby zatrzymać go przyciskiem „**Zatrzymaj Trening**”.

Pełen formularz trenowania został przedstawiony na zdjęciu 6.3.

The screenshot displays the 'Strona Trenowania Modeli' (Model Training Page) with a dark blue background. At the top, there is a navigation bar with tabs: 'Zliczenie', 'Trening' (active), 'Modeli', 'R&A automatyczne oznaczanie zdjęć', 'Tworzenie zbioru danych', 'Benchmark', 'Użytkownicy', 'Historia', and 'Wyjście'. The main content area contains the following elements:

- Nazwa Modelu:** A text input field.
- Ścieżka do danych treningowych:** A text input field with a 'Wybierz dane treningowe' button.
- Ścieżka do danych walidacyjnych:** A text input field with a 'Wybierz dane walidacyjne' button.
- Wybierz Algorytm:** A dropdown menu currently showing 'Mask R-CNN'.
- Wybierz model do doszkolenia (opcjonalnie):** A dropdown menu currently showing 'Nowy model'.
- Liczba epok:** A text input field with the value '10' and a small square icon to its right.
- Współczynnik uczenia:** A text input field with the value '0.000500' and a small square icon to its right.
- Liczba augmentacji na obraz:** A text input field with the value '0' and a small square icon to its right.
- Buttons:** Two large buttons labeled 'Rozpocznij trening' and 'Zatrzymaj trening'.
- Postępy:** A label above a large, empty rectangular box.
- Logi treningu:** A label above a large, empty rectangular box.

Rys. 6.3. Strona Trenowania Modeli

6.4. Strona Modeli

Na **Stronie Modele** przedstawiona jest lista wpisów z bazy danych, zawierająca informacje o wszystkich modelach, które zostały lub są obecnie wytrenowane.

Dla każdego modelu wyświetlane są następujące dane:

- *ID*,
- *Nazwa*,
- *Algorytm*,
- *Wersja*,
- *Dokładność*,
- *Data Utworzenia*,
- *Status*,
- *Data Treningu*.

W przypadku usunięcia modelu, jego pliki zostają usunięte z systemu, natomiast wpis pozostaje w bazie danych ze zmienionym statusem na „**Archiwizowany**”.

Strona przedstawiająca listę modeli została zaprezentowana na zdjęciu 6.4.

Ziczenie	Trening	Modele	Półautomatyczne oznaczanie zdjęć	Tworzenie zbioru danych	Benchmark	Użytkownicy	Historia	Wyjście
ID	Nazwa Modelu	Algorytm	Wersja	Dokładność	Data Utworzenia	Status	Data Treningu	
7	FinalTestMCNNUswanie	MCNN	2.0	None	2025-04-19T15:15:53	deployed	2025-04-19T15:15:53	
8	NowyTestMCNN	MCNN	1.0	None	2025-04-19T15:45:27	archived	2025-04-19T15:45:27	
9	TestZlacz	MCNN	1.0	None	2025-04-19T16:07:01	deployed	2025-04-19T16:07:01	
10	MCNN222	MCNN	1.0	None	2025-04-22T16:04:01	deployed	2025-04-22T16:04:01	
11	POPRWAKIAMCNN	MCNN	1.0	None	2025-04-22T16:08:01	deployed	2025-04-22T16:08:01	
12	TESTMRCNN	Mask R-CNN	1.0	None	2025-04-25T17:17:46	deployed	2025-04-25T17:17:46	
13	10EpochMRCNN	Mask R-CNN	1.0	None	2025-04-26T01:44:12	deployed	2025-04-26T01:44:12	
14	10EpochMCNN	MCNN	1.0	None	2025-04-26T14:57:07	deployed	2025-04-26T14:57:07	
15	334MCNN	MCNN	1.0	None	2025-04-27T16:18:39	deployed	2025-04-27T16:18:39	
16	334MRCNN	Mask R-CNN	1.0	None	2025-04-27T16:26:47	deployed	2025-04-27T16:26:47	
17	AUGMCNN	MCNN	1.0	None	2025-04-28T18:46:05	deployed	2025-04-28T18:46:05	
18	AUGMCNN	MCNN	1.0	None	2025-04-28T19:13:16	deployed	2025-04-28T19:13:16	
19	NAZWATEST	MCNN	1.0	None	2025-04-29T17:39:10	deployed	2025-04-29T17:39:10	
20	NAZWATEST	MCNN	1.0	None	2025-04-29T17:40:35	archived	2025-04-29T17:40:35	
21	TESTNAZWYMCNN	MCNN	1.0	None	2025-04-29T17:44:10	archived	2025-04-29T17:44:10	
22	SOMCNNAUGTEST	MCNN	1.0	None	2025-04-29T20:22:06	deployed	2025-04-29T20:22:06	
23	TESTYWWW	MCNN	1.0	None	2025-05-05T07:47:02	deployed	2025-05-05T07:47:02	
24	pipies_334_v1_FasterRCNN_checkpoint	FasterRCNN	1.0	None	2025-05-05T08:40:03	archived	2025-05-05T08:40:03	
25	pipies	FasterRCNN	1.0	None	2025-05-05T08:43:58	archived	2025-05-05T08:43:58	
26	pipies	FasterRCNN	1.0	None	2025-05-05T09:03:28	archived	2025-05-05T09:03:28	
27	pipies334	FasterRCNN	1.0	None	2025-05-05T09:09:07	archived	2025-05-05T09:09:07	
28	pipies334	FasterRCNN	1.0	None	2025-05-05T09:14:22	archived	2025-05-05T09:14:22	
29	pipies334	FasterRCNN	1.0	None	2025-05-06T17:27:00	archived	2025-05-06T17:27:00	
30	pipies	FasterRCNN	1.0	None	2025-05-06T17:29:43	deployed	2025-05-06T17:29:43	
31	pipiescheckpoint	FasterRCNN	1.0	None	2025-05-06T17:30:49	deployed	2025-05-06T17:30:49	
32	pipies3	FasterRCNN	1.0	None	2025-05-06T17:42:26	deployed	2025-05-06T17:42:26	
33	pipies2	FasterRCNN	1.0	None	2025-05-06T18:30:22	deployed	2025-05-06T18:30:22	
34	MCNN12AUG	MCNN	1.0	None	2025-05-12T14:50:02	deployed	2025-05-12T14:50:02	
35	pipies334v4Tv2	Mask R-CNN	1.0	None	2025-05-17T22:09:36	deployed	2025-05-17T22:09:36	
36	50EpochMRCNN	Mask R-CNN	1.0	None	2025-05-19T08:50:44	deployed	2025-05-19T08:50:44	

Rys. 6.4. Strona Modeli

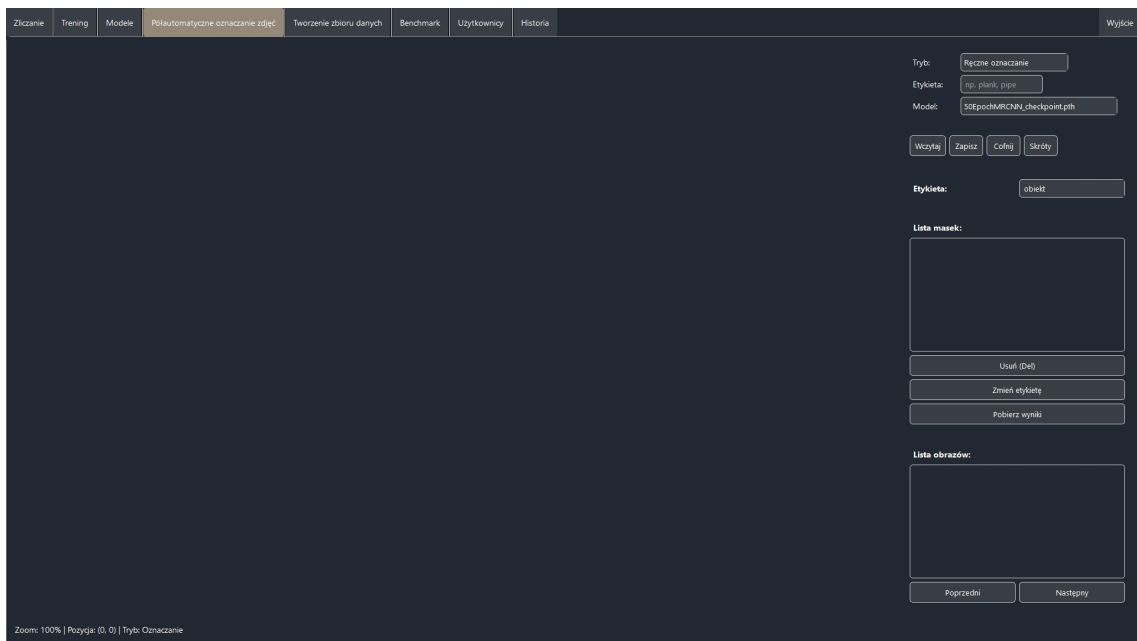
6.5. Strona Oznaczania Zdjęć

Na **Stronie Oznaczania Zdjęć** użytkownik ma możliwość ręcznego oznaczania wczytanych zdjęć w celu przygotowania ich jako danych treningowych dla własnych modeli. Dodatkowo dostępna jest opcja wykorzystania wcześniej wytrenowanego modelu, co pozwala na przyspieszenie całego procesu oznaczania.

Po prawej stronie interfejsu dostępne są następujące opcje:

- *Tryb Oznaczania,*
- *Nazwa Etykiety,*
- *Model,*
- *Lista Masek,*
- *Lista Obrazów.*

Pełny wygląd strony oznaczania został przedstawiony na zdjęciu 6.5.



Rys. 6.5. Strona Oznaczania zdjęć

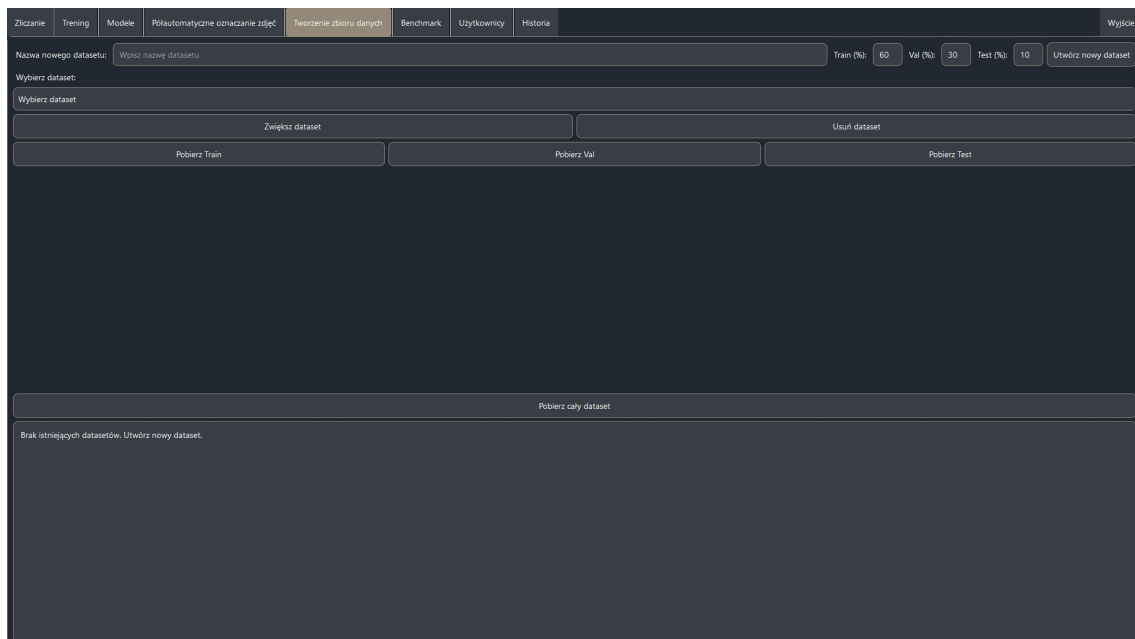
6.6. Strona Tworzenia Datasetu

Na **Stronie Tworzenia Datasetu** użytkownik ma możliwość załadowania zbioru zdjęć, które posłużą jako baza do utworzenia nowego datasetu. Interfejs umożliwia nie tylko wczytanie obrazów, ale również ich odpowiednie przypisanie do jednego z trzech folderów, zgodnie z przeznaczeniem:

- *train* – dane treningowe,
- *val* – dane walidacyjne,
- *test* – dane testowe.

Dodatkowo użytkownik może załączyć odpowiednie adnotacje, które będą stanowić integralną część zbioru danych. Po zakończeniu procesu, utworzony dataset zostaje zapisany pod wcześniej zdefiniowaną nazwą, umożliwiając jego dalsze wykorzystanie do trenowania nowych modeli lub douczania już istniejących.

Pełny wygląd **Strony Tworzenia Datasetu** przedstawiono na rysunku 6.6.



Rys. 6.6. Strona Tworzenia Datasetu

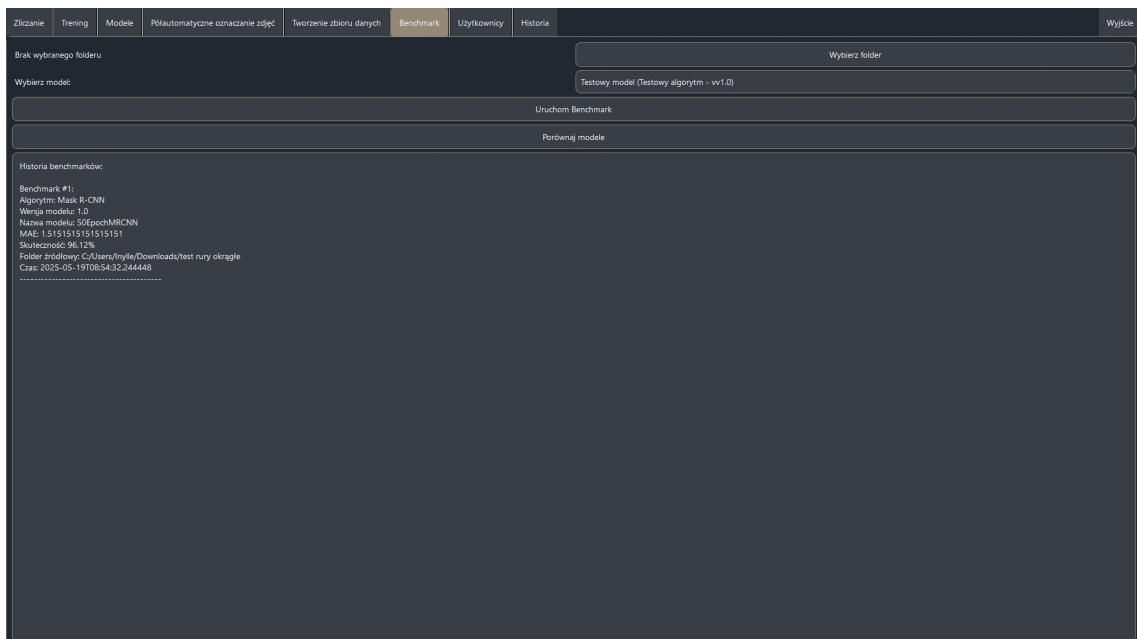
6.7. Strona Benchmark

Na **Stronie Benchmark** użytkownik ma możliwość przetestowania dokładności wytrenowanych modeli na wcześniej przygotowanych *Zbiorach Testowych*. Zbiory te zawierają zdjęcia wraz z adnotacjami, które umożliwiają przeprowadzenie rzetelnej oceny działania modelu.

Po wskazaniu modelu i naciśnięciu przycisku **Uruchom Benchmark**, generowany jest wynik, który zawiera następujące informacje:

- Numer Benchmarku,
- Algorytm,
- Nazwa Modelu,
- MAE (średni błąd bezwzględny),
- Skuteczność,
- Folder Źródłowy,
- Data Benchmarku.

Dodatkowo dostępna jest opcja porównania dwóch modeli. Cała strona została przedstawiona na zdjęciu 6.7.



Rys. 6.7. Strona Benchmark

6.8. Strona Użytkownicy

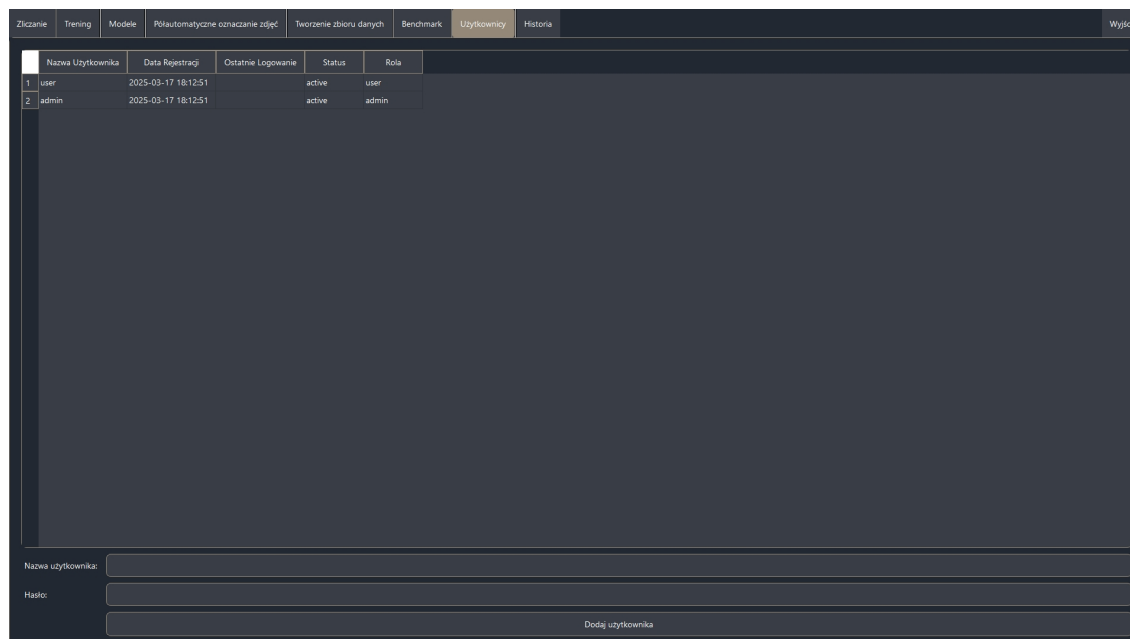
Na **Stronie Użytkownicy**, dostępnej wyłącznie dla *Administratorów*, widoczna jest pełna lista użytkowników zarejestrowanych w bazie danych. Dla każdego z nich prezentowane są następujące informacje:

- Nazwa Użytkownika,
- Data Rejestracji,
- Ostatnie Logowanie,
- Status,
- Rola.

Na dole strony znajduje się również formularz umożliwiający dodanie nowych użytkowników. Składa się on z dwóch pól:

- Nazwa Użytkownika,
- Hasło.

Pełny wygląd tej strony został przedstawiony na zdjęciu 6.8.



Rys. 6.8. Strona Użytkownicy

6.9. Strona Historia

Na **Stronie Historia**, dostępnej wyłącznie dla *Administratorów*, znajduje się lista przedstawiająca akcje wykonane przez użytkowników, które zostały zapisane w bazie danych. Każdy wpis zawiera następujące informacje:

- ID,
- Model,
- Operacja,
- Data,
- Użytkownik.

Rejestrowane operacje dotyczą działań związanych z modelami, takich jak: zliczanie, trenowanie oraz archiwizacja modeli.

Pełny wygląd **Strony Historii** został przedstawiony na zdjęciu 6.9.

Zliczanie	Trening	Model	RLautomatyczne oznaczanie zdjęć	Tworzenie zbioru danych	Benchmark	Użytkownicy	Historia	Wyjście
ID	Model	Operacja	Data	Użytkownik				
681	22	Detekcja obrazu	2025-05-13T21:48:45	2				
682	22	Detekcja obrazu	2025-05-13T21:49:13	2				
683	22	Detekcja obrazu	2025-05-13T21:49:32	2				
684	22	Detekcja obrazu	2025-05-13T21:49:48	2				
685	22	Detekcja obrazu	2025-05-13T21:50:03	2				
686	22	Detekcja obrazu	2025-05-13T22:01:00	2				
687	35	Import	2025-05-17T22:09:36	2				
688	22	Detekcja obrazu	2025-05-19T13:48:54	2				
689	22	Detekcja obrazu	2025-05-19T13:49:33	2				
690	22	Detekcja obrazu	2025-05-19T13:49:49	2				
691	22	Detekcja obrazu	2025-05-19T13:50:59	2				
692	22	Detekcja obrazu	2025-05-19T13:51:15	2				
693	22	Detekcja obrazu	2025-05-19T13:52:06	2				
694	22	Detekcja obrazu	2025-05-19T13:58:03	2				
695	22	Detekcja obrazu	2025-05-19T13:58:16	2				
696	22	Detekcja obrazu	2025-05-19T13:59:10	2				
697	22	Detekcja obrazu	2025-05-19T13:59:28	2				
698	22	Detekcja obrazu	2025-05-19T13:59:57	2				
699	22	Detekcja obrazu	2025-05-19T14:00:10	2				
700	22	Detekcja obrazu	2025-05-19T14:03:04	2				
701	22	Detekcja obrazu	2025-05-19T14:03:17	2				
702	22	Detekcja obrazu	2025-05-19T14:03:29	2				
703	22	Detekcja obrazu	2025-05-19T14:03:45	2				
704	22	Detekcja obrazu	2025-05-19T14:03:58	2				
705	22	Detekcja obrazu	2025-05-19T14:04:13	2				
706	22	Detekcja obrazu	2025-05-19T14:11:10	2				
707	22	Detekcja obrazu	2025-05-19T14:11:57	2				
708	22	Detekcja obrazu	2025-05-19T14:12:14	2				
709	22	Detekcja obrazu	2025-05-19T14:12:27	2				
710	36	Detekcja obrazu	2025-05-19T17:08:39	2				

Rys. 6.9. Strona Historia

7. Czasowa i Zasobowa Analiza Projektu

7.1. Wykresy Burnup Sprintów

Sprint 1 - Tabela Czasowa

Osoba	Łączny czas pracy
Łukasz Nowak	7h 50m
Jakub Piwko	5h 40m
Agnieszka Ryś	3h 30m
Rafał Madoń	7h 30m
Czas całego zespołu	24h 30m

Retrospektywa - Sprint 1

Co poszło dobrze?

- Skuteczna współpraca zespołu – zadania były dobrze rozdzielone.
- Zaplanowanie wyglądu aplikacji i opisanie funkcjonalności
- Utworzenie projektu w Jira

Co można poprawić?

- Potrzeba lepszego oszacowania czasu trwania zadań.
- Część funkcji została zaimplementowana z opóźnieniem z powodu problemów technicznych.
- Logika działania aplikacji zawiera luki w zabezpieczeniach.

Co przeszkadzało?

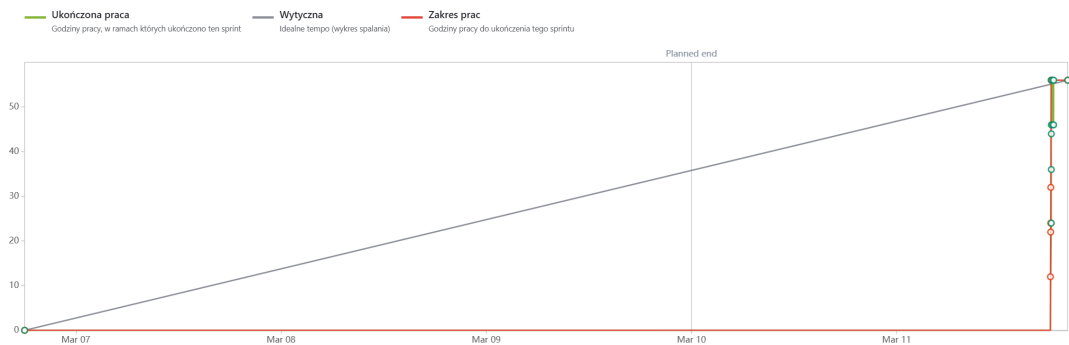
- Problemy sprzętowe (GPU AMD).
- Konflikty zależności w środowisku Docker.
- Brak odpowiednio przygotowanego zbioru danych wejściowych.

Wnioski na przyszłość

- Bardziej obszerna analiza zadania
- Przygotować więcej pytań i wymienić się spostrzeżeniami

Sprint 2 - Tabela Czasowa

Osoba	Łączny czas pracy
Łukasz Nowak	45h
Jakub Piwko	41h
Agnieszka Ryś	37h
Rafał Madoń	44.5h
Czas całego zespołu	167h 30m



Rys. 7.1. Wykres Burnup Sprintu 2

Retrospektywa - Sprint 2

Co poszło dobrze?

- Przygotowanie GUI aplikacji
- Spotkanie organizacyjne i podział zadań
- Standaryzacja przygotowanych danych do nauki modeli

Co można poprawić?

- Oszacowanie czasu do zadań
- Konfiguracja dockera zbyt skomplikowana
- Raport może zawierać więcej aplikacji

Co przeszkadzało?

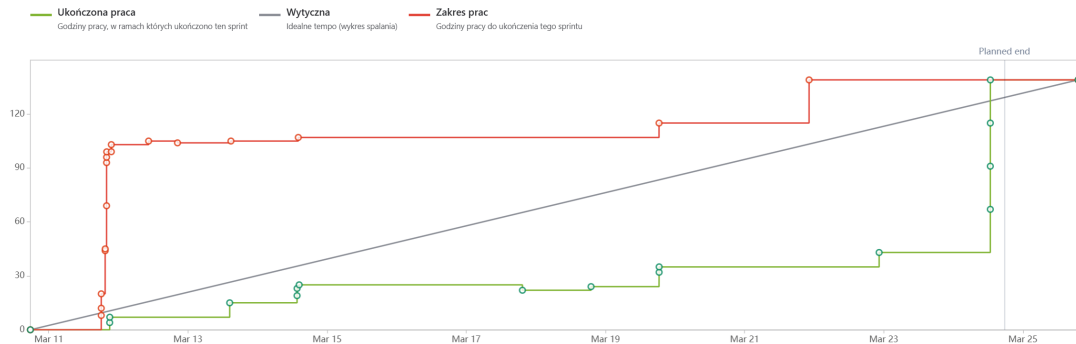
- Brak doświadczenia powoduje przedłużanie się implementacji
- Brak działającego wsparcia od strony sterowników kart graficznych firmy AMD

Wnioski na przyszłość

- Korzystać ze sprawdzonych rozwiązań
- Zwiększyć przewidywany czas na zadania

Sprint 3 - Tabela Czasowa

Osoba	Łączny czas pracy
Łukasz Nowak	13h
Jakub Piwko	16h
Agnieszka Ryś	18h
Rafał Madoń	17h 30m
Czas całego zespołu	64h 30m



Rys. 7.2. Wykres Burnup Sprintu 3

Retrospektywa - Sprint 3

Co poszło dobrze?

- Implementacja algorytmów zliczających elementy na obrazie
- Podział zadań na równe części
- Komunikacja w zespole
- Przygotowanie bazy danych

Co można poprawić?

- Szacowanie czasu nadal zaniżone

Co przeszkadzało?

- Brak czasu na wszystkie czynności związane z uczeniem i przygotowaniem modeli

Wnioski na przyszłość

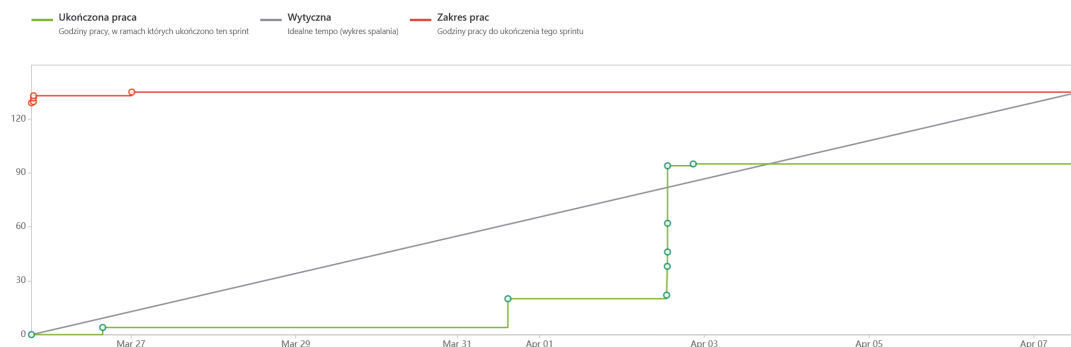
- Skupić się na testowaniu na początku po czym przystąpić do nauki modeli z zadowalającymi wynikami

Sprint 4 - Tabela Czasowa

Pierwotne oszacowanie: 143h

Wykorzystany czas: 192h 30m

Osoba	Łączny czas pracy
Łukasz Nowak	53h
Jakub Piwko	49h
Agnieszka Ryś	47h 30m
Rafał Madoń	43h



Rys. 7.3. Wykres Burnup Sprintu 4

Retrospektywa - Sprint 4

Co poszło dobrze?

- Udało się zintegrować wszystkie główne algorytmy (Mask R-CNN, Faster R-CNN, MCNN) z aplikacją.
- Zespół przeprowadził skuteczną augmentację danych dla wszystkich modeli.
- Zakończono implementację aplikacji mobilnej oraz rozpoczęto jej połączenie z dockerem.

- Implementacja YOLO jako narzędzia do porównania jakości pozwoliła na szybką ocenę pozostałych modeli.

Co można poprawić?

- Nie udało się zrealizować wszystkich zaplanowanych funkcji aplikacyjnych.
- Lepsze zarządzanie czasem — część prac była przeciągnięta do kolejnego sprintu.

Co przeszkadzało?

- Duża złożoność implementowanych algorytmów i ich integracji z aplikacją.
- Brak wystarczającej liczby oznaczonych danych dla niektórych przypadków (np. deski).
- Czasochłonność przygotowania danych i środowisk (docker, mobile).

Wnioski na przyszłość

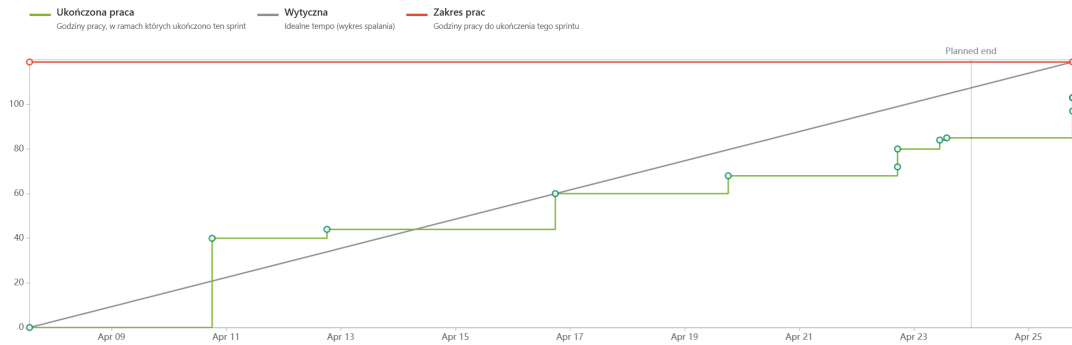
- Podzielić funkcje na mniejsze, szybciej mierzalne cele do realizacji w sprincie.
- Wprowadzić system kontroli jakości predykcji (np. ręczne sprawdzanie etykiet po oznaczeniu).
- Kontynuować rozbudowę zbiorów danych z naciskiem na różnorodność obiektów.
- Wcześniej planować elementy wymagające dużej integracji (np. mobilka, docker, benchmark).

Sprint 5 - Tabela Czasowa

Pierwotne oszacowanie: 127h

Wykorzystany czas: 147h 30m

Osoba	Łączny czas pracy
Łukasz Nowak	45h
Jakub Piwko	39h
Agnieszka Ryś	28h 30m
Rafał Madoń	37h



Rys. 7.4. Wykres Burnup Sprintu 5

Retrospektywa - Sprint 5

Co poszło dobrze?

- Poszerzenie danych testowych i walidacyjnych
- Dodanie funkcji do aplikacji umożliwiającej automatyczne oznaczanie zdjęć
- Dodanie zabezpieczeń przed błędami części programu szkolącej modele
- Mocna poprawa z oszacowaniem czasu

Co można poprawić?

- Wygląd aplikacji jest mało intuicyjny
- Poprawa dokumentacji jest wskazana (zmiany uwzględniające ostatnie implementacje)
- Usunąć niepotrzebne gałęzie z GitHub

Co przeszkadzało?

- Brak czasu na implementację wszystkich zaplanowanych funkcji (np. systemu wyboru najlepszego algorytmu).
- Trudności z implementacją algorytmu SSD oraz potrzebą jego przeprojektowania.
- Złożoność integracji wielu algorytmów oraz ich testowania w aplikacji.

Wnioski na przyszłość

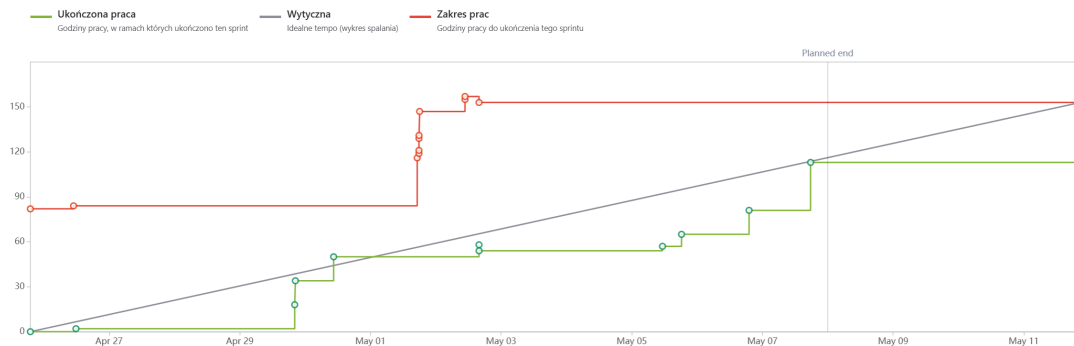
- Lepiej planować zależności między zadaniami, by ograniczyć blokujące się elementy pracy (np. brak implementacji jednego algorytmu wpływa na cały system).
- Wcześniej zaplanować integrację złożonych funkcji (np. auto-labeling, docker, mobilka).
- Przygotować plan awaryjny dla algorytmów, które mogą nie zostać zaimplementowane na czas.

Sprint 6 - Tabela Czasowa

Pierwotne oszacowanie: 175h

Wykorzystany czas: 140h 45m

Osoba	Łączny czas pracy
Łukasz Nowak	28h 45m
Jakub Piwko	29h 15m
Agnieszka Ryś	41h 45m
Rafał Madoń	41h



Rys. 7.5. Wykres Burnup Sprintu 6

Retrospektywa - Sprint 6

Co poszło dobrze?

- Dodanie sporej ilości danych do nauczania modeli
- Dodanie systemu obrabiającego zdjęcie przed liczeniem elementów w niektórych sytuacjach poprawia wynik
- Poprawa jakości Mask oraz Faster RCNN do poziomu 90-96%

- Poprawa wyglądu aplikacji zwiększa wygodę i prostotę używania aplikacji

Co można poprawić?

- Jakość modeli
- Jakość i ilość destylatów modeli do aplikacji mobilnej
- Dokumentacja nie zawiera ostatnich zmian

Co przeszkadzało?

- Mała wiedza na temat wykonywanych zadań (destylacja)
- Złożoność projektu znacząco zwiększona od wstępnych założeń
- Nieporządek w plikach i zależnościach (docker)

Wnioski na przyszłość

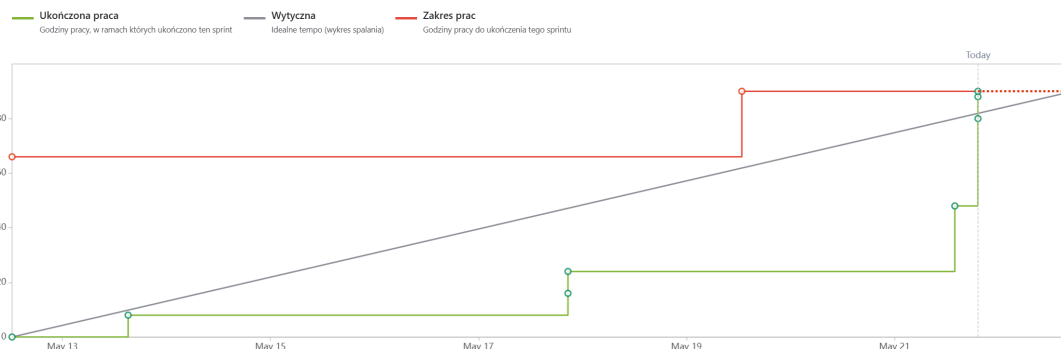
- Poprawa ram założeń projektu
- Zwiększenie czasu na oszacowane zadania
- Częstsza wymiana rodzajów zadań w projekcie

Sprint 7 - Tabela Czasowa

Pierwotne oszacowanie: 230h

Wykorzystany czas: 188h 30m

Osoba	Łączny czas pracy
Łukasz Nowak	49h
Jakub Piwko	33h 30m
Agnieszka Ryś	55h 30m
Rafał Madoń	50h 30m



Rys. 7.6. Wykres Burnup Sprintu 7

Retrospektywa - Sprint 7

Co poszło dobrze?

- Przygotowanie destylatów oraz skryptu do ich tworzenia (znaczące przyspieszenie eksperymentów i integracji)
- Szkolenie modeli i poprawa ich skuteczności — osiągnięto wysoką jakość Mask R-CNN oraz Faster R-CNN
- Dopracowanie i finalizacja dokumentacji oraz pliku README z opisem bibliotek i licencjami
- Znaczna poprawa porządku w repozytorium GitHub i oznaczenie zależności
- Dodanie komentarzy do kodu oraz uporządkowanie struktury projektu
- Dodanie dużej liczby nowych zdjęć do nauki i testów modeli

Co można poprawić?

- Jakość algorytmów

Co przeszkadzało?

- Złożoność projektu znacząco wzrosła względem pierwotnych założeń
- Nieporządek w zależnościach (szczególnie docker + środowiska Python)

Wnioski na przyszłość

- Dokładniejsze oszacowanie czasu trwania sprintu — mimo poprawy, rzeczywisty czas nadal odbiegał od zakładanego
- Lepsze przygotowanie początkowe do skomplikowanych zadań (np. research destylacji przed implementacją)
- Częstsza rotacja zadań między członkami zespołu w celu dywersyfikacji wiedzy i obciążenia

Podsumowanie przewidywanego i rzeczywistego czasu pracy

Sprint	Czas przewidywany	Czas rzeczywisty
Sprint 4	143h	192h 30m
Sprint 5	127h	147h 30m
Sprint 6	175h	140h 45m
Sprint 7	230h	188h 30m
Razem	675h	669h 15m

Podsumowanie czasu pracy zespołu

Osoba	Suma czasu pracy
Łukasz Nowak	240h 30m
Jakub Piwko	213h 30m
Agnieszka Ryś	232h
Rafał Madoń	241h
Czas całego zespołu	927h

7.2. Sposoby komunikacji

Discord

Głosowe spotkania: Odbywały się na kanale głosowym i obejmowały:

- Omawianie aktualnego stanu pracy,
- Rozwiązywanie problemów z zadaniami,
- Narady dotyczące implementacji funkcjonalności,
- Przygotowywanie raportów ze sprintów,
- Podział zadań między członków zespołu,
- Rozwiązywanie konfliktów,
- Wzajemną pomoc członków zespołu.

Komunikacja tekstowa: Za pomocą kanałów tekstowych:

- Przekazywano bieżące informacje o postępach,
- Udostępniano materiały wykorzystywane w projekcie,
- Wymieniano pliki (modele, datasety itp.).

Messenger

Zastosowanie: Platforma wykorzystywana była głównie do:

- Zgłaszania problemów,
- Informowania o postępach w pracy,
- Planowania spotkań,
- Udostępniania krótkich notatek.

8. Przykład Implementacji - Trening

8.1. Trening - API

Metoda `train_model_stream`

Metoda `train_model_stream` w klasie `TrainAPI` odpowiada za uruchomienie procesu treningu modelu detekcji obiektów (np. Mask R-CNN, FasterRCNN, MCNN). Przyjmuje nazwę algorytmu i listę argumentów, które są przekazywane do skryptu treningowego uruchamianego w kontenerze Docker.

Opis działania

- Sprawdza, czy podano obsługiwany algorytm i odpowiednie ścieżki danych treningowych.
- Kopiuje dane treningowe (i walidacyjne) do lokalnego katalogu `data/`, który jest montowany do kontenera.
- Tworzy i filtruje argumenty dla skryptu treningowego.
- Uruchamia proces treningowy w kontenerze Docker z odpowiednim skrypcem i argumentami.
- Odczytuje na bieżąco strumień wyjścia (`stdout` i `stderr`) z procesu i zwraca linie jako generator.
- Obsługuje przerwanie procesu na żądanie użytkownika.
- Sprawdza kod zakończenia procesu, aby wykryć ewentualne błędy.

Kod odpowiadający za wybór algorytmu wskazanego do treningu modelu przedstawiony został na listingu 3 i przedstawia on część tej metody odpowiadającą za dobór algorytmu oraz parametrów treningu.

Kod źródłowy

```

1 def train_model_stream(self, algorithm, *args):
2     if algorithm not in self.train_scripts:
3         logger.error(f"Algorytm {algorithm} nie jest wspierany.")
4         raise RuntimeError(f"Algorytm {algorithm} nie jest
wspierany.")
5
6     self._running = True
7     self.current_algorithm = algorithm
8     script_name = self.train_scripts[algorithm]
9
10    train_dir, val_dir, host_train_path, host_val_path=None, None, None,
None
11    num_augmentations, epochs, model_name = "0", "0", ""
12    for i in range(0, len(args), 2):
13        if args[i] == "--train_dir": train_dir = args[i + 1]
14        elif args[i] == "--host_train_path": host_train_path = ...
15        elif args[i] == "--host_val_path": host_val_path = ...
16        elif args[i] == "--num_augmentations": num_augmentations
...
17        elif args[i] == "--model_name": model_name = args[i + 1]
18        elif args[i] == "--username": self.username = args[i + 1]
19
20    if not train_dir or not host_train_path or not os.path.exists(
host_train_path):
21        logger.error("Niepoprawna ścieżka: %s", host_train_path)
22        raise RuntimeError("Niepoprawna ścieżka.")
23
24    # --- Reszta Metody ---

```

Listing 3. Metoda train_model_stream

Podsumowanie

Metoda `train_model_stream` uruchamia proces treningowy w sposób asynchroniczny i zwraca jego wyjście w czasie rzeczywistym. Obsługuje również kopiowanie danych, przekazywanie argumentów, monitorowanie postępu i obsługę błędów.

8.2. Przykład Implementacji Treningu Mask R-CNN

Parsowanie argumentów

Funkcja `parse_args()` odpowiada za zebranie argumentów linii poleceń, które będą używane do konfiguracji treningu modelu Mask R-CNN. Wśród parametrów znajdują się ścieżki do katalogów treningowego i walidacyjnego, liczba epok, tempo uczenia, liczba augmentacji oraz inne.

```
1 def parse_args():
2     parser = argparse.ArgumentParser(description="Trening Mask R-
3     CNN (v2)")
4     parser.add_argument("--train_dir", type=str, default=
5     DEFAULT_TRAIN_DIR)
6     parser.add_argument("--val_dir", type=str, default=
7     DEFAULT_VAL_DIR)
8     parser.add_argument("--epochs", type=int, default=
9     DEFAULT_EPOCHS)
10    parser.add_argument("--lr", type=float, default=DEFAULT_LR)
11    parser.add_argument("--patience", type=int, default=
12    DEFAULT_PATIENCE)
13    parser.add_argument("--coco_train_path", type=str, default=None)
14    parser.add_argument("--coco_gt_path", type=str, default=None)
15    parser.add_argument("--num_augmentations", type=int, default=
16    DEFAULT_NUM_AUGMENTATIONS)
17    parser.add_argument("--resume", type=str, default=None)
18    parser.add_argument("--model_name", type=str, default=None)
19    return parser.parse_args()
```

Opis: Funkcja zbiera i zwraca argumenty z linii poleceń, co pozwala na elastyczną konfigurację treningu.

Pobranie pretrenowanych wag

Funkcja `download_pretrained_weights()` pobiera pretrenowane wagi dla modelu Mask R-CNN w wersji ResNet50 FPN v2 i zapisuje je lokalnie, co pozwala na późniejsze szybkie ładowanie modelu.

```
1 def download_pretrained_weights():
2     weights = torchvision.models.detection.
      MaskRCNN_ResNet50_FPN_V2_Weights.DEFAULT
3     model = torchvision.models.detection.maskrcnn_resnet50_fpn_v2(
      weights=weights)
4     torch.save(model.state_dict(), PRETRAINED_WEIGHTS_PATH)
```

Opis: Automatycznie pobiera i zapisuje pretrenowane wagi potrzebne do inicjalizacji modelu.

Inicjalizacja modelu

Funkcja `get_model(num_classes, device)` ładuje model Mask R-CNN bez wag (`weights=None`), następnie ładuje wcześniej pobrane pretrenowane wagi i dostosowuje model do liczby klas (w tym klasy tła).

```
1 def get_model(num_classes, device):
2     model = torchvision.models.detection.maskrcnn_resnet50_fpn_v2(
      weights=None)
3     state_dict = torch.load(PRETRAINED_WEIGHTS_PATH, map_location=
      device)
4     model.load_state_dict(state_dict)
5     in_features = model.roi_heads.box_predictor.cls_score.
      in_features
6     model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
      num_classes)
7     model.roi_heads.mask_predictor = MaskRCNNPredictor(in_channels
      =256, dim_reduced=256, num_classes=num_classes)
8     model.to(device)
9     return model
```

Opis: Tworzy model i dopasowuje go do specyfiki zadania (liczba klas), uruchamiając go na wskazanym urządzeniu (CPU/GPU).

Obsługa sygnałów systemowych

Funkcja `signal_handler(sig, frame)` jest prostym handlerem na sygnały systemowe (np. `SIGTERM`), który pozwala na łagodne zakończenie działania programu i czyszczenie zasobów.

```
1 def signal_handler(sig, frame):
2     print(f"Otrzymano sygnał {sig}, czyszczenie zasobów...")
3     sys.exit(0)
```

Opis: Dzięki tej funkcji program reaguje na sygnały systemowe, co jest ważne przy uruchomieniu na serwerach lub w kontenerach.

Główna pętla treningowa

Funkcja `train_model(args, is_api_call=False)` zawiera pełny przebieg treningu: obsługę sygnałów, wybór urządzenia (CPU/GPU), inicjalizację modelu, optymalizatora, schedulerów, loaderów danych, a następnie pętlę treningową i walidacyjną.

```
1 def train_model(args, is_api_call=False):
2     signal.signal(signal.SIGTERM, signal_handler)
3     device = torch.device("cuda" if torch.cuda.is_available() else
4                           "cpu")
5     model = get_model(num_classes=NUM_CLASSES, device=device)
6     optimizer = optim.SGD(model.parameters(), lr=args.lr, momentum=
7                           MOMENTUM, weight_decay=WEIGHT_DECAY)
8     scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='min',
9         factor=SCHEDULER_FACTOR, patience=SCHEDULER_PATIENCE, min_lr=
10        MIN_LR)
11     train_loader, val_loader = get_data_loaders(...)
12     for epoch in range(start_epoch, end_epoch + 1):
13         strata_treningowa = train_one_epoch(model, train_loader,
14             optimizer, device, epoch)
15         strata_walidacyjna, liczba_predykacji, liczba_gt, mAP_bbox,
16         mAP_seg = validate_model(...)
17         scheduler.step(strata_walidacyjna)
```

Opis: Kompleksowy proces treningu z obsługą walidacji, schedulera i ewentualnym zapisem checkpointów.

Generowanie wykresów wyników

Poniższy fragment kodu odpowiada za tworzenie i zapisywanie wykresów z przebiegu treningu, m.in. strat treningowych i walidacyjnych.

```
1 plt.figure(figsize=(10, 5))
2 plt.plot(straty_treningowe, label="Strata treningowa")
3 plt.plot(straty_walidacyjne, label="Strata walidacyjna")
4 plt.savefig(...)
```

Opis: Wizualizacja wyników treningu, ułatwiająca analizę postępów i diagnostykę.

9. Przykład Implementacji - Zliczanie

9.1. Zliczanie - API

Wersje dostępnych modeli

`get_model_versions(algorithm)` Zwraca listę plików modeli dla wybranego algorytmu. Dla **Mask R-CNN** zwracane są tylko pliki z rozszerzeniem `_checkpoint.pth`.

Opis: Umożliwia wybór wersji modelu Mask R-CNN do detekcji.

```
1 def get_model_versions(self, algorithm):
2     if algorithm not in self.algorithms:
3         logger.warning("Algorytm %s nie jest obsługiwany",
4             algorithm)
5         return []
6     model_path = self.algorithms[algorithm]
7     if not model_path.exists():
8         logger.warning("Katalog modeli %s nie istnieje", model_path)
9         return []
10    if algorithm == "Mask R-CNN":
11        models = sorted([file.name for file in model_path.iterdir()
12            if file.is_file() and file.name.endswith('_checkpoint.pth')])
13    else:
14        models = sorted([file.name for file in model_path.iterdir()
15            if file.is_file()])
16    logger.debug("Znaleziono modele dla %s: %s", algorithm, models)
17    return models
```

Ścieżka do konkretnego modelu

`get_model_path(algorithm, version)` Zwraca pełną ścieżkę do wybranego modelu. Dla **Mask R-CNN** akceptowane są tylko pliki kończące się na `_checkpoint.pth`.

Opis: Znajduje model Mask R-CNN gotowy do użycia.

```

1 def get_model_path(self, algorithm, version):
2     if algorithm not in self.algorithms:
3         logger.warning("Algorytm %s nie jest obsługiwany",
4             algorithm)
5         return None
6     model_path = self.algorithms[algorithm] / version
7     if not model_path.exists() or not model_path.is_file():
8         logger.warning("Model %s nie istnieje", model_path)
9         return None
10    if algorithm == "Mask R-CNN":
11        if model_path.name.endswith('_checkpoint.pth'):
12            logger.debug("Znaleziono poprawny model: %s",
13                model_path)
14            return str(model_path)
15        logger.warning("Model Mask R-CNN musi kończyć się na
16            _checkpoint.pth")
17        return None
18    else:
19        return str(model_path)

```

Uruchomienie skryptu detekcji

`run_script(script_name, algorithm, *args)` Uruchamia odpowiedni skrypt detekcji w środowisku backend. Dla **Mask R-CNN** uruchamiany jest skrypt `detect.py`.

Opis: Wykonuje właściwą detekcję za pomocą Mask R-CNN.

```

1 def run_script(self, script_name, algorithm, *args):
2     try:
3         if algorithm == "Mask R-CNN":
4             script_path = f"/app/backend/Mask_RCNN/scripts/{
5                 script_name}"
6             elif algorithm == "MCNN":
7                 script_path = f"/app/backend/MCNN/{script_name}"
8             elif algorithm == "FasterRCNN":
9                 script_path = f"/app/backend/FasterRCNN/{script_name}"
10            else:
11                return f"Błąd: Algorytm {algorithm} nie jest obsł
12                    ugiwany."
13            command = ["python", script_path, *args]

```

```

12     result = subprocess.run(command, capture_output=True, text=
    True, encoding="utf-8", errors="replace")
13     if result.returncode != 0:
14         return f"Błąd: {result.stderr}"
15     return result.stdout
16 except Exception as e:
17     return f"Nieoczekiwany błąd: {e}"

```

Analiza obrazu i liczenie detekcji

`analyze_with_model(image_path, algorithm, version, preprocessing=False)`

- Sprawdza istnienie modelu i obrazu.
- Opcjonalnie wykonuje preprocessing obrazu.
- Uruchamia detekcję Mask R-CNN przy użyciu `run_script`.
- Sprawdza, czy został utworzony wynikowy plik detekcji.
- Wyciąga liczbę detekcji z tekstowego wyniku skryptu (`Detections: (\d+)`).

Opis: Kluczowa metoda do wykrywania obiektów i zliczania wykrytych elementów przy użyciu Mask R-CNN.

```

1 def analyze_with_model(self, image_path, algorithm, version,
    preprocessing=False):
2     model_path = self.get_model_path(algorithm, version)
3     if not model_path:
4         return f"Błąd: Model {version} dla {algorithm} nie istnieje
        .", 0
5     if not os.path.exists(image_path):
6         return f"Błąd: Obraz {image_path} nie istnieje.", 0
7
8     if preprocessing:
9         processed_image_path = preprocess_image(image_path)
10        if not processed_image_path:
11            return f"Błąd podczas preprocessingu obrazu.", 0
12    else:
13        processed_image_path = image_path
14
15    output_file = f"{Path(processed_image_path).stem}_output.txt"
16    output_path = Path(processed_image_path).parent / output_file
17
18    result = self.run_script("detect.py", algorithm,
        processed_image_path, model_path, str(output_path))

```

```
19     logger.debug("Wynik skryptu: %s", result)
20
21     if not output_path.exists():
22         return f"Błąd: Nie znaleziono pliku wynikowego {output_path}
23     }, 0
24
25     with open(output_path, "r") as f:
26         content = f.read()
27         match = re.search(r"Detections: (\d+)", content)
28         detections = int(match.group(1)) if match else 0
29     return str(output_path), detections
```

Podsumowanie kroków detekcji Mask R-CNN

1. Wywołaj `get_model_versions("Mask R-CNN")` – lista dostępnych modeli.
2. Wywołaj `get_model_path("Mask R-CNN", wersja)` – wybór konkretnego modelu.
3. Wywołaj `analyzer_with_model(ścieżka_do_obrazu, "Mask R-CNN", wersja)` – wykonaj detekcję.
4. Otrzymasz wynik: `(ścieżka_do_wyniku, liczba_detekcji)`.

Przykładowy pseudokod

```
1 api = DetectionAPI()
2 available_versions = api.get_model_versions("Mask R-CNN")
3 model_path = api.get_model_path("Mask R-CNN", available_versions
4     [0])
5 result_path, detections = api.analyzer_with_model("image.jpg", "Mask
6     R-CNN", available_versions[0])
7 print(f"Wykryto {detections} obiektów, wynik zapisany w {
8     result_path}")
```

9.2. Przykład Implementacji Zliczania Mask R-CNN

Ładowanie modelu

`load_model(model_path)` Ładuje model Mask R-CNN z checkpointu, konfiguruje odpowiednią liczbę klas, progi NMS i próg detekcji, a następnie przenosi model na GPU/CPU i przełącza w tryb ewaluacji. Opis: Przygotowuje gotowy model do detekcji.

```
1 def load_model(model_path):
2     if not model_path.endswith('_checkpoint.pth'):
3         print(f"Błąd: Ścieżka {model_path} niepoprawna.")
4         sys.exit(1)
5     if not os.path.exists(model_path):
6         print(f"Błąd: Plik {model_path} nie istnieje.")
7         sys.exit(1)
8     model = torchvision.models.detection.maskrcnn_resnet50_fpn_v2(
9         weights="DEFAULT")
10    in_features = model.roi_heads.box_predictor.cls_score.in_features
11    model.roi_heads.box_predictor = torchvision.models.detection.faster_rcnn.FastRCNNPredictor(in_features, NUM_CLASSES)
12    model.roi_heads.mask_predictor = torchvision.models.detection.mask_rcnn.MaskRCNNPredictor(256, 256, NUM_CLASSES)
13    checkpoint = torch.load(model_path, map_location=DEVICE, weights_only=False)
14    model.load_state_dict(checkpoint['model_state_dict'])
15    if isinstance(model.rpn.pre_nms_top_n, dict):
16        model.rpn.pre_nms_top_n["testing"] = NMS_THRESHOLD
17        model.rpn.post_nms_top_n["testing"] = NMS_THRESHOLD
18    model.roi_heads.score_thresh = CONFIDENCE_THRESHOLD
19    model.roi_heads.detections_per_img = DETECTIONS_PER_IMAGE
20    model.to(DEVICE)
21    model.eval()
22    return model
```


Przygotowanie obrazu

`preprocess_image(image_path)` Ładuje obraz, zmienia jego rozmiar do wymaganego przez model, konwertuje na tensor i normalizuje. Opis: Przygotowuje dane wejściowe do detekcji.

```
1 def preprocess_image(image_path):
2     image = Image.open(image_path).convert("RGB")
3     original_size = image.size
4     transform = T.Compose([
5         T.Resize(MODEL_INPUT_SIZE, antialias=True),
6         T.ToImage(),
7         T.ToDtype(torch.float32, scale=True),
8     ])
9     tensor = transform(image).to(DEVICE)
10    return [tensor], image, original_size
```

Wykonanie detekcji

`detect_objects(model, image_tensor_list)` Wykonuje detekcję obiektów na obrazie, zwracając przewidywania (ramki, maski, etykiety, pewności). Opis: Serce detekcji Mask R-CNN.

```
1 def detect_objects(model, image_tensor_list):
2     with torch.no_grad():
3         predictions = model(image_tensor_list)[0]
4     return predictions
```

Rysowanie wyników

`draw_results(image, predictions, original_size)` Rysuje ramki i maski na obrazie, skaluje detekcje do oryginalnego rozmiaru, zwraca obraz NumPy i liczbę detekcji. Opis: Wizualizacja wyników detekcji.

```
1 def draw_results(image, predictions, original_size):
2     image_np = np.array(image)
3     image_np = cv2.resize(image_np, original_size)
4     boxes = predictions['boxes'].cpu().numpy()
5     scores = predictions['scores'].cpu().numpy()
6     labels = predictions['labels'].cpu().numpy()
7     masks = predictions['masks'].cpu().numpy() if 'masks' in
8     predictions else None
9     valid_indices = scores >= CONFIDENCE_THRESHOLD
10    boxes = boxes[valid_indices]
```

```

10     scores = scores[valid_indices]
11     labels = labels[valid_indices]
12     if masks is not None:
13         masks = masks[valid_indices]
14     orig_width, orig_height = original_size
15     model_width, model_height = MODEL_INPUT_SIZE
16     scale_x = orig_width / model_width
17     scale_y = orig_height / model_height
18     scaled_boxes = boxes.copy()
19     scaled_boxes[:, 0] *= scale_x
20     scaled_boxes[:, 1] *= scale_y
21     scaled_boxes[:, 2] *= scale_x
22     scaled_boxes[:, 3] *= scale_y
23     detections_count = 0
24     for i, (box, score, label) in enumerate(zip(scaled_boxes,
scores, labels)):
25         if detections_count >= DETECTIONS_PER_IMAGE:
26             break
27         x1, y1, x2, y2 = box.astype(int)
28         cv2.rectangle(image_np, (x1, y1), (x2, y2), (0, 255, 0), 2)
29         if masks is not None:
30             mask = masks[i, 0] > 0.5
31             mask = cv2.resize(mask.astype(np.uint8), original_size)
32             * 255
33             image_np[mask > 0] = [0, 0, 255]
34             detections_count += 1
35     return image_np, detections_count

```

Zapis wyników

`save_result(image_np, image_path)` Zapisuje obraz z detekcjami do pliku. Opis:
Utrwalenie wyników detekcji.

```

1 def save_result(image_np, image_path):
2     result_image_name = os.path.splitext(os.path.basename(
image_path))[0] + "_detected.jpg"
3     result_path = os.path.join(RESULTS_PATH, result_image_name)
4     cv2.imwrite(result_path, cv2.cvtColor(image_np, cv2.
COLOR_RGB2BGR))
5     return result_path

```

Bibliografia

- [1] Joseph Redmon i in. „You Only Look Once: Unified, Real-Time Object Detection”. W: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016). arXiv:1506.02640, s. 779–788.
- [2] Ultralytics. *YOLO by Ultralytics - Documentation*. <https://docs.ultralytics.com/>. Dostęp: 2025-05-20.
- [3] Git Community. *Git - Distributed Version Control System*. <https://git-scm.com/>. Dostęp: 19.05.2025. 2025.
- [4] GitHub Inc. *GitHub – Where the world builds software*. <https://github.com/>. Dostęp: 19.05.2025. 2025.
- [5] Kaiming He i in. „Mask R-CNN”. W: *IEEE International Conference on Computer Vision (ICCV)* (2017), s. 2961–2969.
- [6] Shaoqing Ren i in. „Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. W: *Advances in Neural Information Processing Systems* 28 (2015).
- [7] Visual Paradigm International. *Visual Paradigm - UML, ERD and Agile Tools*. <https://www.visual-paradigm.com/>. Dostęp: 2025-05-20. 2025.
- [8] Riverbank Computing Limited. *PyQt5 Documentation*. <https://www.riverbankcomputing.com/software/pyqt/intro>. Dostęp: 2025-05-20. 2025.

Spis rysunków

4.1. Oznaczenie wykrytych elementów	10
4.2. Wykres skuteczności uczenia modelu	10
5.1. Diagram przypadków użycia	22
6.1. Formularz Logowania	35
6.2. Strona Zliczania	36
6.3. Strona Trenowania Modeli	37
6.4. Strona Modeli	38
6.5. Strona Oznaczania zdjęć	39
6.6. Strona Tworzenia Datasetu	40
6.7. Strona Benchmark	41
6.8. Strona Użytkownicy	42
6.9. Strona Historia	43
7.1. Wykres Burnup Sprintu 2	45
7.2. Wykres Burnup Sprintu 3	46
7.3. Wykres Burnup Sprintu 4	47
7.4. Wykres Burnup Sprintu 5	49
7.5. Wykres Burnup Sprintu 6	50
7.6. Wykres Burnup Sprintu 7	51

Spis tabel

5.1. Scenariusz 1 - Rejestracja	23
5.2. Scenariusz 2 - Logowanie	24
5.3. Scenariusz 3 - Trenowanie nowego modelu	25
5.4. Scenariusz 4 - Douczanie modelu	26
5.5. Scenariusz 5 - Importowanie modelu	27
5.6. Scenariusz 6 - Usuwanie modelu	28
5.7. Scenariusz 7 - Zliczanie elementów na podstawie modelu	29
5.8. Scenariusz 8 - Ustawienie statusu użytkownika	30
5.9. Scenariusz 9 - Wyświetlenie historii operacji w systemie	31
5.10. Scenariusz 10 - Tworzenie Datasetu	32
5.11. Scenariusz 11 - Automatyczne oznaczanie zdjęć	33
5.12. Scenariusz 12 - Benchmark modelu	34

Spis listingów

1.	Kod odpowiedzialny za trenowanie	8
2.	Kod odpowiedzialny za znajdowanie elementów	9
3.	Metoda train_model.stream	56